MODUL 03 · WIDGET & UI

Arsitektur Widget & Lifecycle Antarmuka

Membedah filosofi 'Everything is a Widget', anatomi MaterialApp/Scaffold/AppBar, widget dasar (Text, Icon, Image, Container, Padding, SizedBox, Align, Card, ListTile), StatelessWidget vs StatefulWidget beserta setState dan lifecycle, cara memecah UI menjadi widget kecil dengan parameter dan callback, const & key, serta tema Material 3 dengan mode gelap.

Durasi Pertemuan 3 (150 Menit) + Belajar Mandiri

Prasyarat Modul 02: Fondasi Dart 3

Capaian Pembelajaran

- Menjelaskan relasi Widget Tree, Element Tree, dan RenderObject Tree.
- Menyusun kerangka aplikasi dengan MaterialApp, Scaffold, dan AppBar.
- Menggunakan widget dasar Text, Icon, Image (asset & network), Container, Padding, SizedBox, Center, Align, Card, dan ListTile.
- Memilih dengan tepat kapan menggunakan StatelessWidget vs StatefulWidget, serta memakai setState dengan benar.
- Menganalisis siklus hidup State (initState, didChangeDependencies, build, didUpdateWidget, dispose) dan membersihkan resource di dispose.
- Memecah UI menjadi widget kecil yang menerima parameter dan callback.
- Menerapkan const, Key, dan ThemeData Material 3 (ColorScheme.fromSeed, textTheme, mode gelap).

Daftar Isi Modul

1. Filosofi 'Everything is a Widget' & Tiga Pohon Flutter
2. Anatomi Aplikasi: MaterialApp, Scaffold, & AppBar
3. Text & TextStyle
4. Icon & IconButton
5. Image: Asset & Network
6. Container & BoxDecoration (Studi Kasus Kartu Mahasiswa)
7. Padding, SizedBox, Center, & Align
8. Card & ListTile
9. StatelessWidget vs StatefulWidget
10. StatefulWidget & setState: Counter Lengkap
11. Lifecycle State dengan Demo Log
12. Memecah Widget: Parameter & Callback
13. const & Key
14. ThemeData Material 3 & Mode Gelap
15. Tugas Praktikum
16. Kuis Pemahaman & Kunci Jawaban

1. Filosofi 'Everything is a Widget' & Tiga Pohon Flutter

Di dalam Flutter, hampir segala sesuatu adalah **Widget**. Bukan hanya tombol atau gambar, tetapi struktur layout (Padding, Center), tema warna (Theme), hingga pendeteksi sentuhan (GestureDetector) semuanya adalah widget.

Di balik layar, Flutter mengelola tiga pohon paralel untuk mencapai performa render 60-120 FPS yang mulus:



Widget adalah "resep", bukan tampilan

Bayangkan widget seperti **resep masakan**. Resep hanya berisi keterangan: bahan apa, berapa banyak, dimasak bagaimana. Flutter yang "memasak" resep itu menjadi piksel di layar. Karena widget hanya resep, membuat ulang widget itu murah sekali. Flutter boleh membuat ulang ribuan widget setiap detik tanpa membuat aplikasi lambat.

Widget disusun bersarang seperti pohon. Setiap widget punya *parent* (induk) dan bisa punya *child* (satu anak) atau *children* (banyak anak). Lihat contoh kecil berikut:

```
// Pohon widget sederhana: Center -> Column -> [Icon, Text]
Center(
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: const [
      Icon(Icons.school, size: 48),
      Text('Halo, Mahasiswa UNSOED!'),
    ],
  ),
)
```



Pohon widget Center → Column → Icon + Text

Jika digambar sebagai pohon, susunannya menjadi seperti ini:

```
Center
 └── Column
      ├── Icon (school)
      └── Text ("Halo, Mahasiswa UNSOED!")
```

Program Flutter paling kecil

Program Flutter selalu dimulai dari fungsi `main()` yang memanggil `runApp()`. Widget yang diberikan ke `runApp()` menjadi akar (root) pohon widget. Coba jalankan kode ini untuk melihat bahwa satu widget `Text` pun sudah bisa menjadi aplikasi.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  // runApp menerima satu widget sebagai akar pohon widget.
  runApp(
    const Center(
      child: Text(
        'Halo, Flutter!',
        // Tanpa MaterialApp, arah teks wajib ditentukan sendiri.
        textDirection: TextDirection.ltr,
        style: TextStyle(fontSize: 32, color: Colors.white),
      ),
    ),
```

```
  );
}
```



Program terkecil: satu Text tanpa MaterialApp, latar hitam

Hasilnya adalah layar hitam dengan tulisan putih di tengah. Perhatikan bahwa kita harus menulis `textDirection` sendiri karena belum ada `MaterialApp` yang menyediakan pengaturan bawaan (arah teks, tema, font). Di section berikutnya kita akan memakai `MaterialApp` supaya hal-hal seperti ini diurus otomatis.

> 💡 **Cara membaca kode Flutter**
>
> Baca kode widget dari luar ke dalam: widget paling luar adalah pembungkus, lalu turun ke `child` atau `children`. Biasakan memberi koma di akhir setiap argumen (*trailing comma*) agar format otomatis (`Shift+Alt+F` di VS Code) menyusun kode menjadi bertingkat rapi.

⚠️ **Kesalahan umum**

- Lupa memanggil `runApp()` di `main()` sehingga layar tetap kosong.
- Menganggap widget sebagai objek yang "diubah-ubah". Widget bersifat *immutable* (tidak bisa diubah). Untuk mengubah tampilan, Flutter membuat widget baru dengan data baru.
- Salah menaruh kurung tutup `)` sehingga widget masuk ke induk yang salah. Gunakan trailing comma dan format otomatis.

2. Anatomi Aplikasi: MaterialApp, Scaffold, & AppBar

Hampir semua aplikasi Flutter yang kamu buat akan memakai tiga widget kerangka: `MaterialApp` (aplikasi), `Scaffold` (halaman), dan `AppBar` (bilah judul). Memahami peran ketiganya membuat kode lain jauh lebih mudah dibaca.

Widget	Analogi	Tugas utama
`MaterialApp`	Gedung kampus	Menyediakan tema, judul aplikasi, navigasi, bahasa, dan arah teks untuk seluruh halaman.
`Scaffold`	Satu ruang kelas	Kerangka satu halaman: tempat untuk `appBar`, `body`, `floatingActionButton`, `drawer`, dan `bottomNavigationBar`.
`AppBar`	Papan nama di pintu kelas	Bilah di bagian atas berisi judul, tombol kembali, dan tombol aksi.

Contoh kecil: kerangka paling minimal

```
MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: const Text('Beranda')),
    body: const Center(child: Text('Isi halaman')),
  ),
)
```



Kerangka minimal: MaterialApp + Scaffold + AppBar

Dari luar ke dalam: `MaterialApp` membungkus `Scaffold` lewat parameter `home`. `Scaffold` punya slot `appBar` dan `body`. Slot `body` adalah area utama di bawah AppBar.

Contoh lengkap: semua slot Scaffold

Kode berikut memperlihatkan slot-slot penting `Scaffold` sekaligus. Jalankan, lalu coba tekan ikon menu (garis tiga) dan tombol bulat di kanan bawah.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const AplikasiKampus());
}

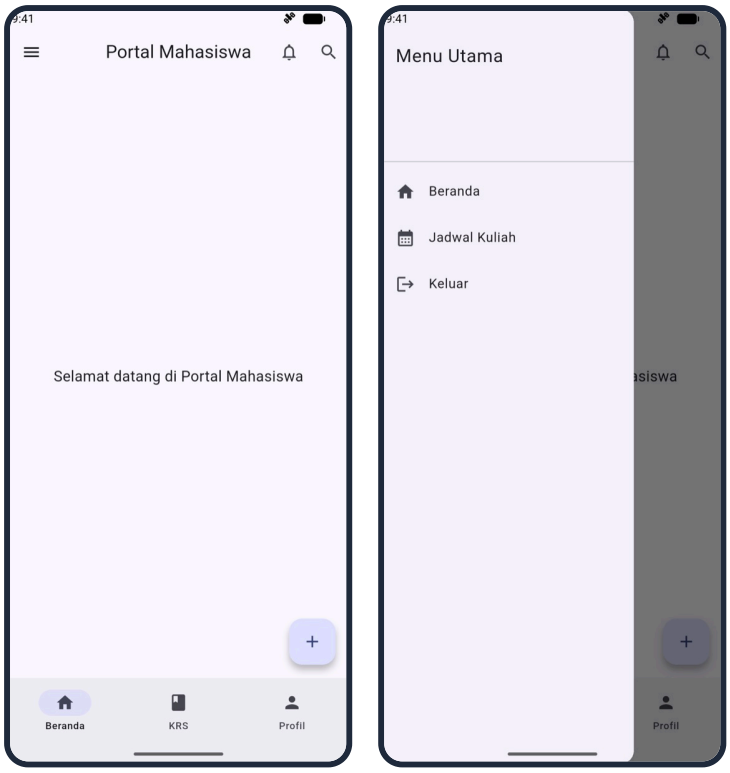
class AplikasiKampus extends StatelessWidget {
  const AplikasiKampus({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Portal UNSOED', // nama aplikasi di task manager / tab browser
      debugShowCheckedModeBanner: false, // sembunyikan pita "DEBUG"
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const HalamanBeranda(),
    );
  }
}
```

```dart
class HalamanBeranda extends StatelessWidget {
  const HalamanBeranda({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // 1. AppBar: bilah judul di atas
      appBar: AppBar(
        title: const Text('Portal Mahasiswa'),
        centerTitle: true,
        actions: [
          IconButton(
            icon: const Icon(Icons.notifications_outlined),
            tooltip: 'Notifikasi',
            onPressed: () {},
          ),
          IconButton(
            icon: const Icon(Icons.search),
            tooltip: 'Cari',
            onPressed: () {},
          ),
        ],
      ),
      // 2. Drawer: menu samping. Ikon menu muncul otomatis di AppBar.
      drawer: Drawer(
        child: ListView(
          children: const [
            DrawerHeader(child: Text('Menu Utama', style: TextStyle(fontSize: 22))),
            ListTile(leading: Icon(Icons.home), title: Text('Beranda')),
            ListTile(leading: Icon(Icons.calendar_month), title: Text('Jadwal Kuliah')),
            ListTile(leading: Icon(Icons.logout), title: Text('Keluar')),
          ],
        ),
      ),
      // 3. Body: area utama halaman
      body: const Center(
        child: Text(
          'Selamat datang di Portal Mahasiswa',
          style: TextStyle(fontSize: 18),
        ),
      ),
      // 4. Tombol aksi melayang di kanan bawah
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        tooltip: 'Tambah',
        child: const Icon(Icons.add),
      ),
      // 5. Navigasi bawah (Material 3)
      bottomNavigationBar: NavigationBar(
        selectedIndex: 0,
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home), label: 'Beranda'),
          NavigationDestination(icon: Icon(Icons.book), label: 'KRS'),
          NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    );
```

```
  }
}
```



Semua slot Scaffold: AppBar, body, FAB, NavigationBar

Setelah ikon menu ditekan: Drawer terbuka

Penjelasan kode

- `runApp(const AplikasiKampus())` menjadikan `AplikasiKampus` sebagai akar aplikasi.
- `MaterialApp` mengatur `title`, `theme`, dan halaman pertama lewat `home`. Kita memisahkan halaman ke class `HalamanBeranda` supaya rapi.
- `AppBar.actions` menerima daftar widget (biasanya `IconButton`) yang tampil di sisi kanan.
- Begitu `drawer` diisi, `Scaffold` otomatis menambahkan ikon menu di kiri AppBar.
- `NavigationBar` adalah navigasi bawah gaya Material 3. Di sini indeksnya masih tetap 0; cara mengubahnya dengan `setState` dibahas di section StatefulWidget dan Modul 07.

Properti AppBar yang sering dipakai

Properti	Fungsi
`title`	Widget judul, biasanya `Text`.
`centerTitle`	`true` agar judul di tengah (default Android: kiri, iOS: tengah).
`leading`	Widget di kiri judul (misalnya tombol kembali kustom).
`actions`	Daftar tombol di kanan.
`backgroundColor` / `foregroundColor`	Warna latar dan warna teks/ikon AppBar.
`bottom`	Widget di bawah AppBar, misalnya `TabBar`.

⚠️ **Kesalahan umum**

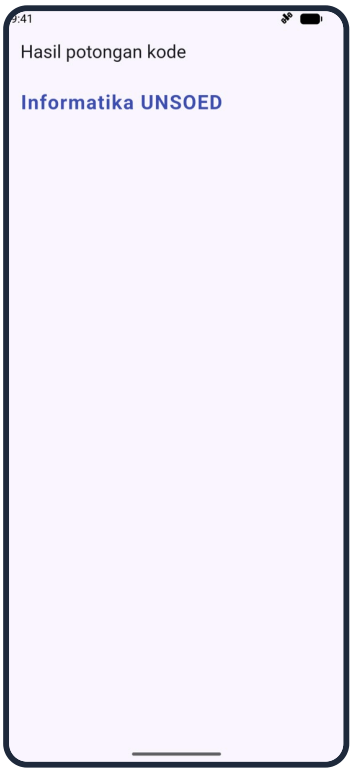
- **Membuat `MaterialApp` di setiap halaman.** Cukup satu `MaterialApp` di akar aplikasi. Setiap halaman cukup memakai `Scaffold`.
- **Lupa `Scaffold`.** Jika `Text` langsung ditaruh di `home` tanpa `Scaffold`, teks tampil dengan garis bawah kuning dan latar hitam karena tidak ada `Material` di atasnya.
- **Menaruh widget besar langsung di `appBar`.** Slot `appBar` mengharapkan `PreferredSizeWidget` seperti `AppBar`, bukan `Container` biasa.

3. Text & TextStyle

Widget `Text` menampilkan tulisan. Tampilan tulisan (ukuran, warna, ketebalan, jarak huruf) diatur lewat objek `TextStyle` yang diberikan ke parameter `style`.

Contoh kecil

```
const Text(
  'Informatika UNSOED',
  style: TextStyle(
    fontSize: 24,                // ukuran huruf (logical pixel)
    fontWeight: FontWeight.bold, // ketebalan: w100 s.d. w900
    color: Colors.indigo,        // warna teks
    letterSpacing: 1.2,          // jarak antar huruf
  ),
)
```



Text dengan TextStyle: besar, tebal, berwarna

Properti penting

Milik	Properti	Fungsi
`Text`	`textAlign`	Perataan: `TextAlign.left`, `center`, `right`, `justify`.
`Text`	`maxLines`	Batas jumlah baris.
`Text`	`overflow`	Perilaku jika teks kepanjangan, misalnya `TextOverflow.ellipsis` (menambah "...").
`TextStyle`	`fontStyle`	`FontStyle.italic` untuk huruf miring.
`TextStyle`	`decoration`	Garis bawah, coret: `TextDecoration.underline`, `lineThrough`.
`TextStyle`	`height`	Tinggi baris sebagai kelipatan ukuran font (misal 1.5).

Contoh lengkap: berbagai gaya teks

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const DemoTeks(),
    );
  }
}

class DemoTeks extends StatelessWidget {
  const DemoTeks({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Text & TextStyle')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
```

```dart
// 1. Teks polos tanpa gaya
const Text('1. Teks polos tanpa style'),
const SizedBox(height: 12),

// 2. Ukuran, tebal, warna
const Text(
  '2. Judul Besar',
  style: TextStyle(
    fontSize: 28,
    fontWeight: FontWeight.w800,
    color: Colors.teal,
  ),
),
const SizedBox(height: 12),

// 3. Miring, garis bawah, coret
const Text(
  '3. Miring dan bergaris bawah',
  style: TextStyle(
    fontStyle: FontStyle.italic,
    decoration: TextDecoration.underline,
  ),
),
const Text(
  'Harga lama: Rp150.000',
  style: TextStyle(decoration: TextDecoration.lineThrough, color: Colors.grey),
),
const SizedBox(height: 12),

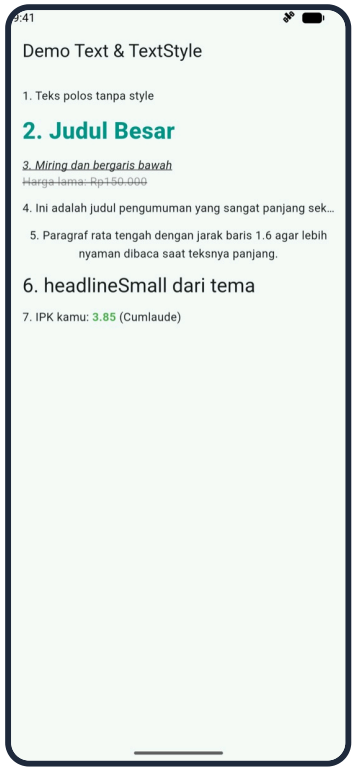
// 4. Teks panjang dipotong dengan ellipsis
const Text(
  '4. Ini adalah judul pengumuman yang sangat panjang sekali sehingga '
  'tidak muat dalam satu baris layar ponsel mahasiswa',
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
),
const SizedBox(height: 12),

// 5. Rata tengah dan jarak baris
const Text(
  '5. Paragraf rata tengah dengan jarak baris 1.6 agar '
  'lebih nyaman dibaca saat teksnya panjang.',
  textAlign: TextAlign.center,
  style: TextStyle(height: 1.6),
),
const SizedBox(height: 12),

// 6. Memakai gaya dari tema (disarankan)
Text(
  '6. headlineSmall dari tema',
  style: Theme.of(context).textTheme.headlineSmall,
),
const SizedBox(height: 12),

// 7. Text.rich: satu kalimat, beberapa gaya
const Text.rich(
  TextSpan(
    text: '7. IPK kamu: ',
    children: [
```

```
                TextSpan(
                  text: '3.85',
                  style: TextStyle(fontWeight: FontWeight.bold, color: Colors.green),
                ),
                TextSpan(text: ' (Cumlaude)'),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```



Berbagai gaya teks, ellipsis, dan Text.rich

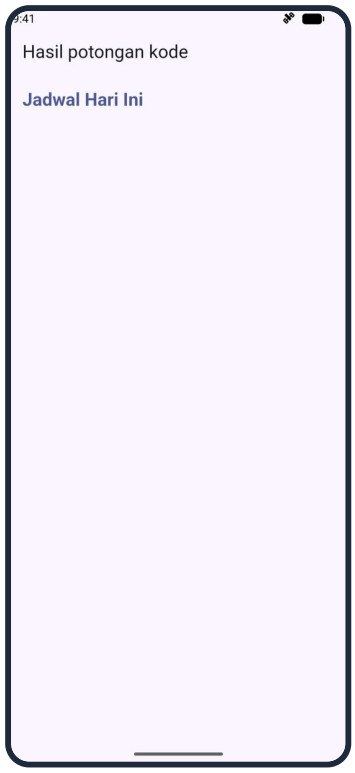
Penjelasan kode

- Teks panjang di Dart bisa ditulis sebagai beberapa string yang ditempel berurutan (*adjacent strings*), seperti pada nomor 4 dan 5. Dart menggabungkannya otomatis.
- `maxLines: 1` + `overflow: TextOverflow.ellipsis` adalah pasangan yang paling sering dipakai untuk judul di daftar.
- `Theme.of(context).textTheme.headlineSmall` mengambil gaya teks bawaan tema Material 3. Cara ini membuat ukuran huruf konsisten di seluruh aplikasi.
- `Text.rich` dengan `TextSpan` dipakai jika satu kalimat butuh beberapa gaya berbeda.

Menggabungkan gaya tema dengan `copyWith`

Sering kita ingin gaya dari tema, tetapi warnanya diganti. Gunakan `copyWith` agar tidak menulis ulang semua properti:

```dart
Text(
  'Jadwal Hari Ini',
  style: Theme.of(context).textTheme.titleLarge?.copyWith(
        color: Theme.of(context).colorScheme.primary,
        fontWeight: FontWeight.bold,
      ),
)
```



Gaya titleLarge dari tema yang diubah dengan copyWith

Tanda `?.` dipakai karena `titleLarge` bertipe `TextStyle?` (boleh null).

⚠️ Kesalahan umum

- Memberi angka bertipe `int` ke tempat yang butuh `double` sebenarnya aman (`fontSize: 24` boleh), tetapi menulis `fontWeight: 700` salah. Gunakan `FontWeight.w700`.
- Memakai `overflow: TextOverflow.ellipsis` tanpa `maxLines` atau tanpa batas lebar. Ellipsis hanya muncul jika teks memang dibatasi.
- Menulis ukuran font berbeda-beda secara manual di setiap halaman. Lebih baik pakai `textTheme` agar konsisten.

4. Icon & IconButton

Widget `Icon` menampilkan simbol dari font ikon. Flutter sudah membawa ribuan ikon Material lewat class `Icons`, jadi kamu tidak perlu mengunduh gambar apa pun.

Contoh kecil

```dart
const Icon(
  Icons.favorite,      // jenis ikon
  size: 32,            // ukuran (default 24)
  color: Colors.red,   // warna
  semanticLabel: 'Favorit', // dibacakan oleh pembaca layar (aksesibilitas)
)
```



Icon favorit merah ukuran 32

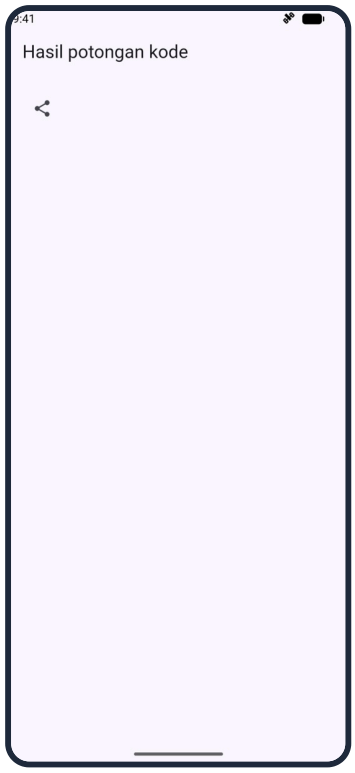
Nama ikon bisa dicari di fonts.google.com/icons. Hampir setiap ikon punya beberapa varian: `Icons.home` (terisi), `Icons.home_outlined` (garis), `Icons.home_rounded`, dan `Icons.home_sharp`.

Icon vs IconButton

`Icon` hanya gambar dan **tidak bisa diklik**. Jika ikon harus bisa ditekan, pakai `IconButton` yang memiliki `onPressed` dan efek sentuhan.

```dart
IconButton(
  icon: const Icon(Icons.share),
  tooltip: 'Bagikan',
  onPressed: () {
    debugPrint('Tombol bagikan ditekan');
```

```
  },
)
```



IconButton berbentuk ikon bagikan

Contoh lengkap: menu ikon portal kampus

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

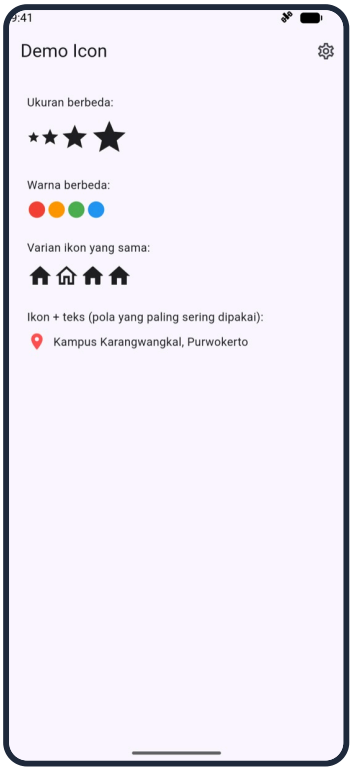
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DemoIkon(),
    );
  }
}

class DemoIkon extends StatelessWidget {
  const DemoIkon({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Demo Icon'),
        actions: [
          IconButton(
            icon: const Icon(Icons.settings_outlined),
            tooltip: 'Pengaturan',
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Pengaturan ditekan')),
              );
            },
          ),
        ],
      ),
      body: const Padding(
        padding: EdgeInsets.all(24),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Ukuran berbeda:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.star, size: 16),
                Icon(Icons.star, size: 24),
                Icon(Icons.star, size: 36),
                Icon(Icons.star, size: 48),
              ],
            ),
            SizedBox(height: 24),
            Text('Warna berbeda:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.circle, color: Colors.red),
                Icon(Icons.circle, color: Colors.orange),
                Icon(Icons.circle, color: Colors.green),
                Icon(Icons.circle, color: Colors.blue),
              ],
            ),
            SizedBox(height: 24),
            Text('Varian ikon yang sama:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.home, size: 32),
                Icon(Icons.home_outlined, size: 32),
                Icon(Icons.home_rounded, size: 32),
                Icon(Icons.home_sharp, size: 32),
              ],
            ),
            SizedBox(height: 24),
            Text('Ikon + teks (pola yang paling sering dipakai):'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.location_on, color: Colors.redAccent),
                SizedBox(width: 8),
```

```
                Text('Kampus Karangwangkal, Purwokerto'),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```



Ukuran, warna, dan varian ikon

Penjelasan kode

- Seluruh isi `body` bisa `const` karena tidak ada nilai yang berubah. Satu `const` di depan `Padding` sudah berlaku untuk semua anak di dalamnya.
- `IconButton` di AppBar memakai `ScaffoldMessenger.of(context).showSnackBar(...)` untuk menampilkan pesan singkat di bawah layar.
- Pola **ikon + `SizedBox` + teks** di dalam `Row` akan sangat sering kamu pakai.

⚠️ **Kesalahan umum**

- Membungkus `Icon` dengan `GestureDetector` agar bisa diklik. Bisa, tetapi tidak ada efek sentuhan dan area sentuhnya kecil. Gunakan `IconButton`.
- Salah nama ikon (misalnya `Icons.homes`) akan langsung error saat kompilasi. Cek ejaan di situs Google Fonts Icons.
- Lupa `tooltip` pada `IconButton`. Tooltip membantu pengguna dan pembaca layar memahami fungsi tombol.

5. Image: Asset & Network

Widget `Image` menampilkan gambar. Ada dua sumber yang paling sering dipakai: **asset** (file gambar yang ikut dibundel di dalam aplikasi) dan **network** (gambar dari internet lewat URL).

Konstruktor	Sumber	Kapan dipakai
`Image.asset('assets/...')`	File di folder proyek	Logo, ilustrasi, ikon kustom yang selalu sama.
`Image.network('https://...')`	Internet	Foto profil, gambar berita, data dari server.

Langkah 1: siapkan folder asset

Buat folder `assets/images/` di akar proyek (sejajar dengan `lib/` dan `pubspec.yaml`), lalu salin gambar ke sana, misalnya `logo_unsoed.png`.

```
nama_proyek/
├── assets/
|   └── images/
|       └── logo_unsoed.png
├── lib/
|   └── main.dart
└── pubspec.yaml
```

Langkah 2: daftarkan di `pubspec.yaml`

Buka `pubspec.yaml`, cari bagian `flutter:`, lalu tambahkan `assets:`. Perhatikan indentasi: `assets:` menjorok 2 spasi di bawah `flutter:`, dan daftar di bawahnya menjorok 4 spasi.

`pubspec.yaml` **(potongan)**

```yaml
flutter:
  uses-material-design: true

  # Daftarkan seluruh isi folder (akhiri dengan garis miring)
  assets:
    - assets/images/
```

Setelah menyimpan `pubspec.yaml`, jalankan perintah berikut lalu **hentikan dan jalankan ulang** aplikasi (hot reload tidak cukup untuk asset baru):

```
flutter pub get
flutter run
```

Langkah 3: tampilkan gambar

```
Image.asset(
  'assets/images/logo_unsoed.png',
  width: 120,
  height: 120,
  fit: BoxFit.contain, // gambar utuh, tidak terpotong
)
```

Gambar dari internet

```
Image.network(
  'https://picsum.photos/id/1/400/250',
  height: 180,
  width: double.infinity,
  fit: BoxFit.cover, // penuhi kotak, boleh terpotong
)
```

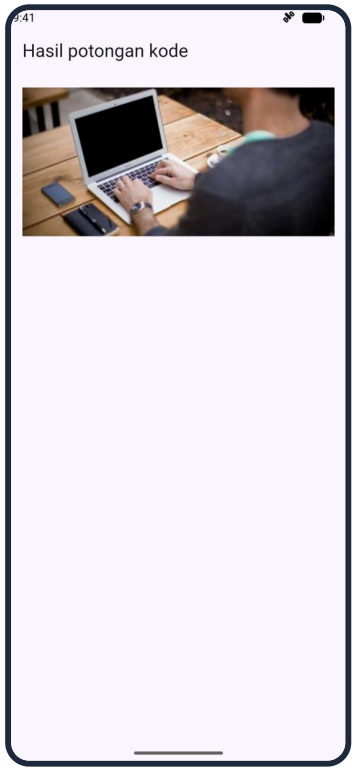


Image.network dengan BoxFit.cover

> ℹ️ **Izin internet di Android**
>
> Saat debug, izin internet sudah aktif. Untuk build rilis Android, pastikan `android/app/src/main/AndroidManifest.xml` berisi `<uses-permission android:name="android.permission.INTERNET"/>`. Detailnya dibahas di Modul 09 dan 16.

Nilai `BoxFit` yang perlu diketahui

Nilai	Hasil
`BoxFit.cover`	Memenuhi kotak, sisi yang berlebih dipotong. Cocok untuk banner dan foto.
`BoxFit.contain`	Gambar utuh di dalam kotak, mungkin ada ruang kosong. Cocok untuk logo.
`BoxFit.fill`	Ditarik memenuhi kotak, gambar bisa gepeng.
`BoxFit.fitWidth` / `fitHeight`	Menyesuaikan lebar atau tinggi saja.

Contoh lengkap: gambar network dengan loading dan error

Contoh ini hanya memakai gambar network supaya langsung bisa dijalankan tanpa menyiapkan asset. Gambar dari internet butuh waktu dimuat dan bisa gagal (misalnya saat offline), jadi kita tampilkan indikator loading dan ikon cadangan.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

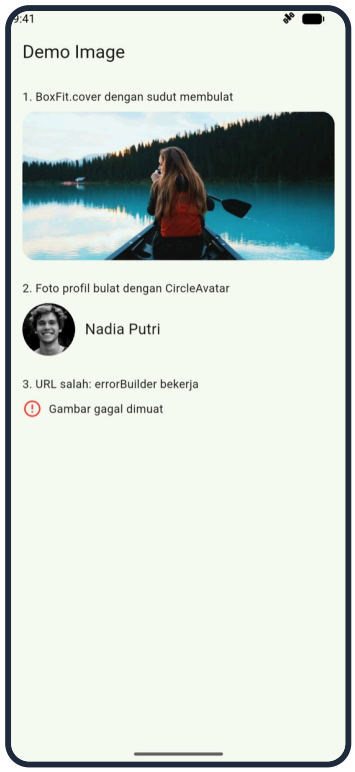
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const DemoGambar(),
    );
  }
}

class DemoGambar extends StatelessWidget {
  const DemoGambar({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Image')),
      body: ListView(
        padding: const EdgeInsets.all(16),
```

```
        children: [
          const Text('1. BoxFit.cover dengan sudut membulat'),
          const SizedBox(height: 8),
          // ClipRRect memotong anak menjadi sudut membulat
          ClipRRect(
            borderRadius: BorderRadius.circular(16),
            child: Image.network(
              'https://picsum.photos/id/1011/600/300',
              height: 180,
              width: double.infinity,
              fit: BoxFit.cover,
              // Dipanggil berulang selama gambar diunduh
              loadingBuilder: (context, child, progress) {
                if (progress == null) return child; // selesai dimuat
                return const SizedBox(
                  height: 180,
                  child: Center(child: CircularProgressIndicator()),
                );
              },
              // Dipanggil jika gambar gagal dimuat
              errorBuilder: (context, error, stackTrace) {
                return Container(
                  height: 180,
                  color: Colors.grey.shade300,
                  alignment: Alignment.center,
                  child: const Icon(Icons.broken_image, size: 48),
                );
              },
            ),
          ),
          const SizedBox(height: 24),
          const Text('2. Foto profil bulat dengan CircleAvatar'),
          const SizedBox(height: 8),
          Row(
            children: [
              CircleAvatar(
                radius: 32,
                foregroundImage: const NetworkImage('https://i.pravatar.cc/150?img=12'),
                // child tampil jika foregroundImage belum/gagal dimuat
                child: const Icon(Icons.person),
              ),
              const SizedBox(width: 12),
              const Text('Nadia Putri', style: TextStyle(fontSize: 18)),
            ],
          ),
          const SizedBox(height: 24),
          const Text('3. URL salah: errorBuilder bekerja'),
          const SizedBox(height: 8),
          Image.network(
            'https://contoh.invalid/gambar-tidak-ada.png',
            height: 100,
            errorBuilder: (context, error, stackTrace) => const Row(
              children: [
                Icon(Icons.error_outline, color: Colors.red),
                SizedBox(width: 8),
                Text('Gambar gagal dimuat'),
              ],
            ),
          ),
```

```
        ],
      ),
    );
  }
}
```



Gambar network dengan sudut membulat, CircleAvatar, dan errorBuilder

Penjelasan kode

- `loadingBuilder` menerima `progress`. Jika `null`, artinya gambar sudah selesai diunduh sehingga kita kembalikan `child` (gambar aslinya).
- `errorBuilder` mencegah layar merah dan menggantinya dengan tampilan cadangan.
- `ClipRRect` memotong gambar agar sudutnya membulat. `Image` sendiri tidak punya properti `borderRadius`.
- `CircleAvatar` dengan `foregroundImage` tetap menampilkan ikon `child` jika foto gagal dimuat.

Versi asset dari contoh di atas

Setelah langkah 1 dan 2 selesai, kamu bisa mengganti `Image.network(...)` dengan asset, dan `NetworkImage` dengan `AssetImage`:

```
Column(
  children: [
    Image.asset('assets/images/logo_unsoed.png', height: 100),
    const SizedBox(height: 12),
    const CircleAvatar(
      radius: 32,
      backgroundImage: AssetImage('assets/images/logo_unsoed.png'),
```

```
    ),
  ],
)
```

⚠️ **Kesalahan umum**

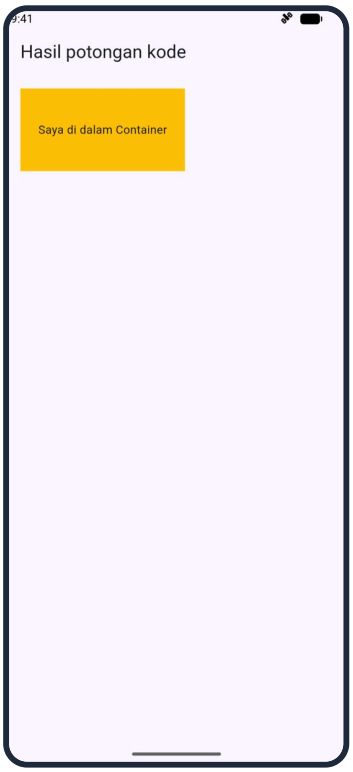
- **"Unable to load asset"**: path di kode berbeda dengan nama file (huruf besar/kecil berpengaruh), atau folder belum didaftarkan di `pubspec.yaml`.
- **Indentasi YAML salah**: `assets:` harus berada di dalam `flutter:` (2 spasi), bukan di tingkat paling kiri.
- **Hanya hot reload** setelah menambah asset. Lakukan `flutter pub get` dan jalankan ulang aplikasi.
- **Gambar network tanpa** `errorBuilder`: saat internet mati, pengguna melihat area kosong atau pesan error.

6. Container & BoxDecoration (Studi Kasus Kartu Mahasiswa)

`Container` adalah "kotak serbaguna". Satu widget ini bisa mengatur ukuran (`width`, `height`), jarak dalam (`padding`), jarak luar (`margin`), posisi anak (`alignment`), dan dekorasi (`decoration`: warna, sudut membulat, border, bayangan, gradasi).

Contoh kecil: kotak berwarna

```
Container(
  width: 200,
  height: 100,
  color: Colors.amber,
  alignment: Alignment.center,
  child: const Text('Saya di dalam Container'),
)
```



Container kuning 200 × 100 dengan teks di tengah

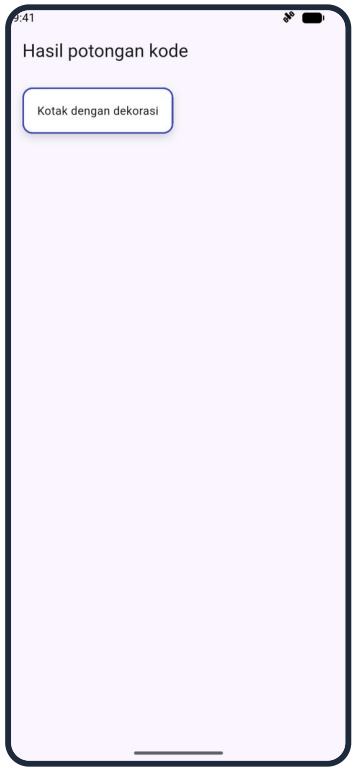
Padding vs margin

```
┌──────────────── margin (jarak ke luar) ────────────────┐
|  ┌──────────── batas Container (warna) ──────────┐     |
|  |   padding (jarak ke dalam)                    |     |
|  |      ┌──────── child ────────┐                |     |
|  |      |  Text('Halo')         |                |     |
|  |      └───────────────────────┘                |     |
|  └───────────────────────────────────────────────┘     |
└────────────────────────────────────────────────────────┘
```

`EdgeInsets` dipakai untuk keduanya: `EdgeInsets.all(16)` (semua sisi), `EdgeInsets.symmetric(horizontal: 16, vertical: 8)`, dan `EdgeInsets.only(left: 8, top: 4)`.

BoxDecoration: menghias kotak

```
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),          // sudut membulat
    border: Border.all(color: Colors.indigo, width: 2), // garis tepi
    boxShadow: [
      BoxShadow(
        color: Colors.black.withValues(alpha: 0.15),  // bayangan transparan
        blurRadius: 8,
        offset: const Offset(0, 4),                   // geser ke bawah 4px
      ),
    ],
  ),
  child: const Text('Kotak dengan dekorasi'),
)
```



BoxDecoration: border, sudut membulat, bayangan

Contoh lengkap: galeri dekorasi Container

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
```

```dart
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const GaleriContainer(),
    );
  }
}

class GaleriContainer extends StatelessWidget {
  const GaleriContainer({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Galeri Container')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Warna polos + ukuran tetap
          Container(
            height: 80,
            color: Colors.indigo.shade100,
            alignment: Alignment.center,
            child: const Text('1. Warna polos'),
          ),
          const SizedBox(height: 16),

          // 2. Sudut membulat + border
          Container(
            height: 80,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(16),
              border: Border.all(color: Colors.indigo, width: 2),
            ),
            child: const Text('2. Border + sudut membulat'),
          ),
          const SizedBox(height: 16),

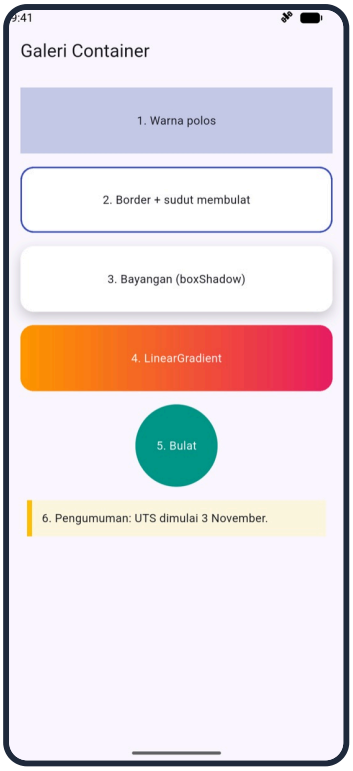
          // 3. Bayangan
          Container(
            height: 80,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(16),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withValues(alpha: 0.2),
                  blurRadius: 12,
```

```
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: const Text('3. Bayangan (boxShadow)'),
        ),
        const SizedBox(height: 16),

        // 4. Gradasi warna
        Container(
          height: 80,
          alignment: Alignment.center,
          decoration: BoxDecoration(
            gradient: const LinearGradient(
              colors: [Colors.orange, Colors.pink],
              begin: Alignment.centerLeft,
              end: Alignment.centerRight,
            ),
            borderRadius: BorderRadius.circular(16),
          ),
          child: const Text('4. LinearGradient', style: TextStyle(color: Colors.white)),
        ),
        const SizedBox(height: 16),

        // 5. Lingkaran
        Center(
          child: Container(
            width: 100,
            height: 100,
            alignment: Alignment.center,
            decoration: const BoxDecoration(
              color: Colors.teal,
              shape: BoxShape.circle, // jangan dipakai bersama borderRadius
            ),
            child: const Text('5. Bulat', style: TextStyle(color: Colors.white)),
          ),
        ),
        const SizedBox(height: 16),

        // 6. Border hanya di sisi kiri (gaya kartu pengumuman)
        Container(
          padding: const EdgeInsets.all(12),
          margin: const EdgeInsets.symmetric(horizontal: 8),
          decoration: BoxDecoration(
            color: Colors.amber.shade50,
            border: const Border(
              left: BorderSide(color: Colors.amber, width: 6),
            ),
          ),
          child: const Text('6. Pengumuman: UTS dimulai 3 November.'),
        ),
      ],
    ),
  );
 }
}
```



Galeri dekorasi Container

Penjelasan kode

- `alignment: Alignment.center` menaruh `child` di tengah kotak.
- `Colors.indigo.shade100` adalah tingkat terang warna (50, 100, ... 900).
- `withValues(alpha: 0.2)` membuat warna 20% tidak transparan. Ini pengganti `withOpacity` yang sudah usang.
- `BoxShape.circle` membuat kotak menjadi lingkaran; ukurannya sebaiknya persegi (lebar = tinggi).
- `Border(left: BorderSide(...))` membuat garis hanya di satu sisi.

Studi kasus: Kartu Tanda Mahasiswa

Berikut implementasi kode kartu mahasiswa yang memadukan `Container`, `BoxDecoration`, `Text`, `Icon`, dan `SizedBox`:

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      title: 'Kartu Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text("Kartu Tanda Mahasiswa")),
        body: const Column(
          children: [
            SizedBox(height: 16),
            KartuMahasiswaWidget(
              nama: "Nadia Putri",
              nim: "21102001",
              jurusan: "S1 Informatika",
            ),
            KartuMahasiswaWidget(
              nama: "Fikri Ramadhan",
              nim: "21102045",
              jurusan: "S1 Sistem Informasi",
              isAktif: false,
            ),
          ],
        ),
      ),
    );
  }
}

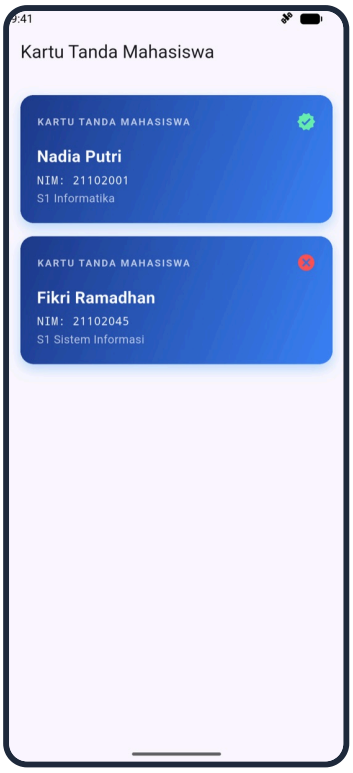
class KartuMahasiswaWidget extends StatelessWidget {
  final String nama;
  final String nim;
  final String jurusan;
  final bool isAktif;

  const KartuMahasiswaWidget({
    super.key,
    required this.nama,
    required this.nim,
    required this.jurusan,
    this.isAktif = true,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(20),
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      decoration: BoxDecoration(
        gradient: const LinearGradient(
          colors: [Color(0xFF1E3A8A), Color(0xFF3B82F6)], // Gradasi Biru Kampus
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
        ),
        borderRadius: BorderRadius.circular(16),
        boxShadow: [
          BoxShadow(
            // withValues menggantikan withOpacity yang sudah deprecated
            color: Colors.blue.withValues(alpha: 0.3),
            blurRadius: 12,
```

```
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              // Flexible mencegah overflow jika layar HP sempit
              const Flexible(
                child: Text(
                  "KARTU TANDA MAHASISWA",
                  overflow: TextOverflow.ellipsis,
                  style: TextStyle(
                    color: Colors.white70,
                    fontSize: 12,
                    letterSpacing: 1.5,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
              Icon(
                isAktif ? Icons.verified : Icons.cancel,
                color: isAktif ? Colors.greenAccent : Colors.redAccent,
              ),
            ],
          ),
          const SizedBox(height: 16),
          Text(
            nama,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 20,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 4),
          Text(
            "NIM: $nim",
            style: const TextStyle(
              color: Colors.white,
              fontFamily: 'monospace',
              fontSize: 14,
            ),
          ),
          const SizedBox(height: 2),
          Text(
            jurusan,
            style: const TextStyle(
              color: Colors.white70,
              fontSize: 14,
            ),
          ),
        ],
      ),
    );
```

```
  }
}
```



Studi kasus Kartu Tanda Mahasiswa (aktif dan tidak aktif)

Penjelasan studi kasus

- `KartuMahasiswaWidget` menerima data lewat konstruktor (`nama`, `nim`, `jurusan`, `isAktif`), sehingga satu class bisa dipakai untuk banyak mahasiswa.
- `width: double.infinity` membuat kartu selebar mungkin sesuai ruang dari induknya.
- Ikon berubah berdasarkan `isAktif` memakai operator ternary `kondisi ? a : b`.

> ⚠️ **Kesalahan umum**
>
> - **Memakai `color` dan `decoration` bersamaan.** Ini menimbulkan error *"Cannot provide both a color and a decoration"*. Jika memakai `decoration`, pindahkan warna ke `BoxDecoration(color: ...)`.
> - **`shape: BoxShape.circle` + `borderRadius`** sekaligus akan error. Pilih salah satu.
> - **Memakai Container hanya untuk jarak.** Jika hanya butuh jarak, `SizedBox` atau `Padding` lebih ringan dan lebih jelas maksudnya (dibahas di section berikutnya).

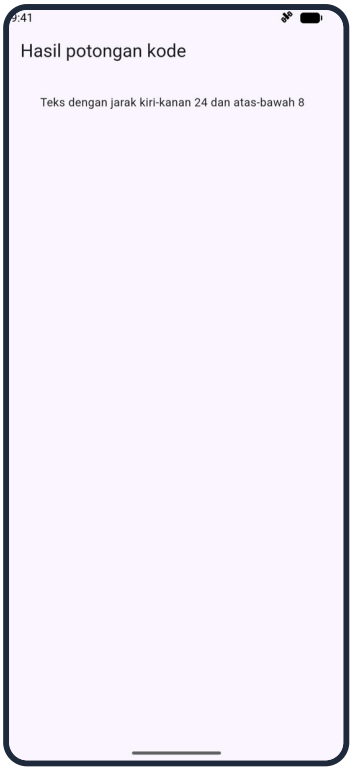
7. Padding, SizedBox, Center, & Align

`Container` memang serbaguna, tetapi untuk tugas tunggal Flutter menyediakan widget yang lebih kecil dan jelas maksudnya: **`Padding`** (jarak dalam), **`SizedBox`** (ukuran tetap atau jarak kosong), **`Center`** (menengahkan), dan **`Align`** (menaruh anak di posisi tertentu).

Widget	Fungsi	Contoh
`Padding`	Memberi ruang di sekeliling anak	`Padding(padding: EdgeInsets.all(16), child: ...)`
`SizedBox`	Kotak berukuran pasti, atau jarak kosong	`SizedBox(height: 12)`
`Center`	Menaruh anak tepat di tengah	`Center(child: ...)`
`Align`	Menaruh anak di titik mana pun	`Align(alignment: Alignment.topRight, child: ...)`

Padding

```
const Padding(
  padding: EdgeInsets.symmetric(horizontal: 24, vertical: 8),
  child: Text('Teks dengan jarak kiri-kanan 24 dan atas-bawah 8'),
)
```



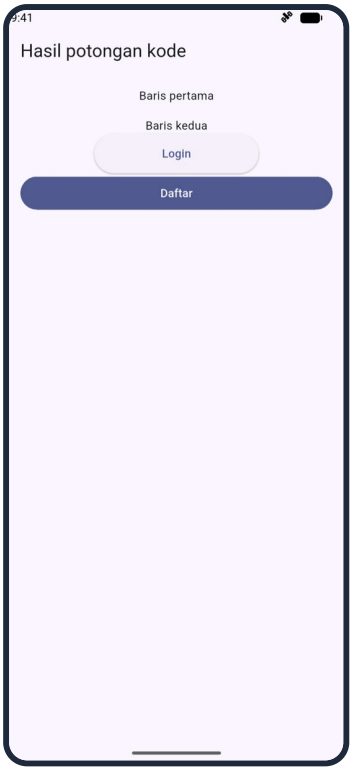
Padding horizontal 24 dan vertikal 8

SizedBox: tiga kegunaan

```
Column(
  children: [
    // 1. Jarak vertikal antar widget
    const Text('Baris pertama'),
    const SizedBox(height: 16),
    const Text('Baris kedua'),

    // 2. Memaksa ukuran anak
    SizedBox(
      width: 200,
      height: 48,
      child: ElevatedButton(onPressed: () {}, child: const Text('Login')),
    ),

    // 3. Selebar mungkin
    SizedBox(
      width: double.infinity,
      child: FilledButton(onPressed: () {}, child: const Text('Daftar')),
    ),
  ],
)
```



SizedBox sebagai jarak, ukuran tetap, dan selebar layar

Align dan nilai `Alignment`

`Alignment(x, y)` memakai koordinat dari `-1` sampai `1`. Titik `(0, 0)` adalah tengah, `(-1, -1)` pojok kiri atas, dan `(1, 1)` pojok kanan bawah.

```
(-1,-1) topLeft      (0,-1) topCenter      (1,-1) topRight
(-1, 0) centerLeft   (0, 0) center         (1, 0) centerRight
(-1, 1) bottomLeft   (0, 1) bottomCenter   (1, 1) bottomRight
```

Contoh lengkap: membandingkan keempat widget

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const DemoTataLetakDasar(),
    );
```

```
  }
}

class DemoTataLetakDasar extends StatelessWidget {
  const DemoTataLetakDasar({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Padding, SizedBox, Center, Align')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text('Padding:'),
          Container(
            color: Colors.deepPurple.shade50,
            child: const Padding(
              padding: EdgeInsets.all(24),
              child: Text('Ada ruang 24 px di sekeliling teks ini'),
            ),
          ),
          const SizedBox(height: 24),

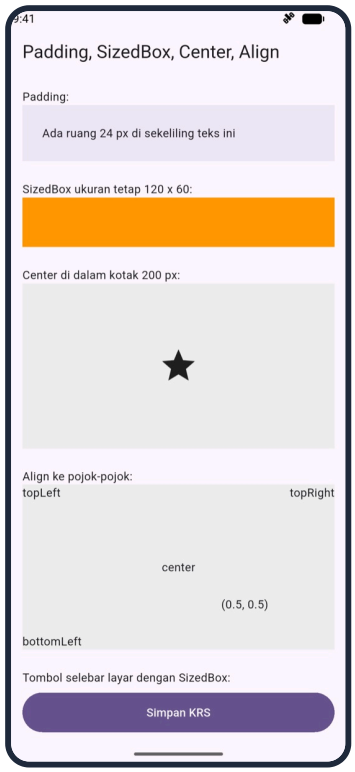
          const Text('SizedBox ukuran tetap 120 x 60:'),
          const SizedBox(
            width: 120,
            height: 60,
            child: ColoredBox(color: Colors.orange),
          ),
          const SizedBox(height: 24),

          const Text('Center di dalam kotak 200 px:'),
          Container(
            height: 200,
            color: Colors.grey.shade200,
            child: const Center(child: Icon(Icons.star, size: 48)),
          ),
          const SizedBox(height: 24),

          const Text('Align ke pojok-pojok:'),
          Container(
            height: 200,
            color: Colors.grey.shade200,
            child: const Stack(
              children: [
                Align(alignment: Alignment.topLeft, child: Text('topLeft')),
                Align(alignment: Alignment.topRight, child: Text('topRight')),
                Align(alignment: Alignment.center, child: Text('center')),
                Align(alignment: Alignment.bottomLeft, child: Text('bottomLeft')),
                Align(alignment: Alignment(0.5, 0.5), child: Text('(0.5, 0.5)')),
              ],
            ),
          ),
          const SizedBox(height: 24),

          const Text('Tombol selebar layar dengan SizedBox:'),
          const SizedBox(height: 8),
          SizedBox(
            width: double.infinity,
```

```
            height: 48,
            child: FilledButton(
              onPressed: () {},
              child: const Text('Simpan KRS'),
            ),
          ),
        ],
      ),
    );
  }
}
```



Perbandingan Padding, SizedBox, Center, dan Align

Penjelasan kode

- `Container` berwarna di sekitar `Padding` membuat jarak dalam terlihat jelas.
- `ColoredBox` adalah widget ringan untuk memberi warna latar saja.
- `Stack` dipakai agar beberapa `Align` bisa menempati kotak yang sama (Stack dibahas lengkap di Modul 04).
- `Alignment(0.5, 0.5)` menunjukkan bahwa posisi tidak harus di sudut; nilai pecahan juga boleh.

💡 **Pilih widget yang paling sederhana**

Butuh jarak? `SizedBox`. Butuh ruang di sekitar? `Padding`. Butuh warna + sudut + bayangan sekaligus? Baru `Container`. Kode jadi lebih mudah dibaca dan bisa `const`.

⚠️ **Kesalahan umum**

- Menulis `SizedBox(height: 16)` di dalam `Row` untuk memberi jarak horizontal. Di `Row` yang dipakai adalah `width`.
- Menaruh `Center` di dalam `Column` lalu heran anaknya tidak di tengah layar secara vertikal. `Center` hanya menengahkan di dalam ruang yang diberikan induknya. Gunakan `mainAxisAlignment` pada `Column` (Modul 04).
- Lupa `const` pada `SizedBox` dan `Padding` yang isinya tetap. Linter akan memberi saran *prefer_const_constructors*.

8. Card & ListTile

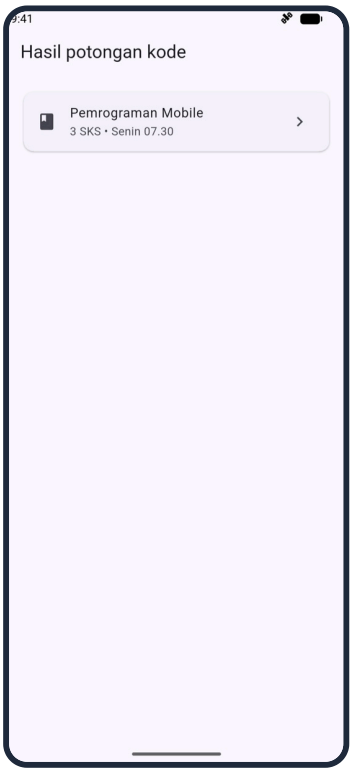
`Card` adalah panel bersudut membulat dengan bayangan/warna permukaan khas Material. `ListTile` adalah satu baris berisi ikon/gambar di kiri, judul, subjudul, dan widget di kanan. Keduanya sering dipakai bersama untuk daftar data seperti mata kuliah, pengumuman, atau kontak.

Anatomi ListTile

```
┌──────────────────────────────────────────────────┐
| [leading]  title (judul)                [trailing] |
|            subtitle (keterangan)                  |
└──────────────────────────────────────────────────┘
```

Contoh kecil

```dart
Card(
  child: ListTile(
    leading: const Icon(Icons.book),
    title: const Text('Pemrograman Mobile'),
    subtitle: const Text('3 SKS • Senin 07.30'),
    trailing: const Icon(Icons.chevron_right),
    onTap: () {
      debugPrint('Mata kuliah dipilih');
    },
  ),
)
```



Card berisi ListTile lengkap

Varian Card di Material 3

Material 3 menyediakan tiga gaya: `Card()` (elevated, sedikit bayangan), `Card.filled()` (warna permukaan, tanpa bayangan), dan `Card.outlined()` (garis tepi).

Contoh lengkap: daftar mata kuliah

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const DaftarMatkul(),
    );
  }
}

class DaftarMatkul extends StatelessWidget {
  const DaftarMatkul({super.key});

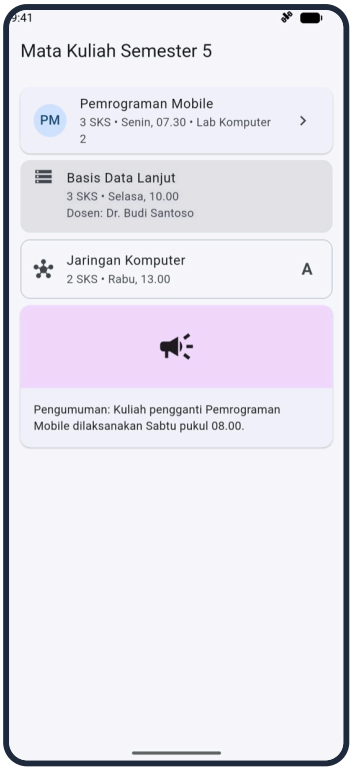
  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Semester 5')),
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          // 1. Card biasa + ListTile lengkap
          Card(
            child: ListTile(
              leading: CircleAvatar(
                backgroundColor: warna.primaryContainer,
                child: const Text('PM'),
              ),
              title: const Text('Pemrograman Mobile'),
              subtitle: const Text('3 SKS • Senin, 07.30 • Lab Komputer 2'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Membuka Pemrograman Mobile')),
                );
              },
            ),
          ),
```

```
          // 2. Card.filled + subtitle dua baris
          const Card.filled(
            child: ListTile(
              leading: Icon(Icons.storage),
              title: Text('Basis Data Lanjut'),
              subtitle: Text('3 SKS • Selasa, 10.00\nDosen: Dr. Budi Santoso'),
              isThreeLine: true,
            ),
          ),

          // 3. Card.outlined + trailing berupa teks
          const Card.outlined(
            child: ListTile(
              leading: Icon(Icons.hub),
              title: Text('Jaringan Komputer'),
              subtitle: Text('2 SKS • Rabu, 13.00'),
              trailing: Text('A', style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
            ),
          ),

          // 4. Card berisi konten bebas (bukan ListTile)
          Card(
            clipBehavior: Clip.antiAlias, // agar isi ikut terpotong sudutnya
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Container(
                  height: 100,
                  color: warna.tertiaryContainer,
                  alignment: Alignment.center,
                  child: const Icon(Icons.campaign, size: 48),
                ),
                const Padding(
                  padding: EdgeInsets.all(16),
                  child: Text(
                    'Pengumuman: Kuliah pengganti Pemrograman Mobile '
                    'dilaksanakan Sabtu pukul 08.00.',
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```



Varian Card Material 3 dan Card berisi konten bebas

Penjelasan kode

- `Theme.of(context).colorScheme` memberi akses ke warna tema (misal `primaryContainer`), jadi warna otomatis serasi.
- `onTap` membuat seluruh baris bisa ditekan dan memunculkan efek riak (*ripple*).
- `isThreeLine: true` memberi tinggi ekstra saat subtitle berisi dua baris.
- `clipBehavior: Clip.antiAlias` diperlukan agar `Container` berwarna di dalam Card tidak keluar dari sudut membulat.

> ⚠️ **Kesalahan umum**
>
> - Menaruh `Row` panjang di `title` sehingga overflow. Gunakan `Text` dengan `maxLines` dan `overflow`.
> - Menaruh widget lebar tanpa batas (misal `TextField` atau `Row` tanpa `Expanded`) di `trailing`. Slot `trailing` ukurannya terbatas.
> - Lupa `clipBehavior` sehingga gambar/warna di dalam Card menutupi sudut membulat.

9. StatelessWidget vs StatefulWidget

Setiap widget buatan sendiri adalah turunan dari salah satu dari dua class: `StatelessWidget` (tampilan tidak berubah sendiri) atau `StatefulWidget` (punya data yang bisa berubah dan membuat tampilan digambar ulang).

Aspek	StatelessWidget	StatefulWidget
Karakteristik	Statis, tidak berubah sepanjang waktu	Dinamis, dapat berubah berdasarkan interaksi atau data
State	Tidak memiliki objek State internal	Memiliki objek `State<T>` yang persisten
Jumlah class	1 class	2 class: widget + State-nya
Kapan Digunakan?	Label teks statis, icon, layout struktur, kartu profil statis	Formulir input, checkbox toggle, animasi, data hasil request API

Pertanyaan penentu

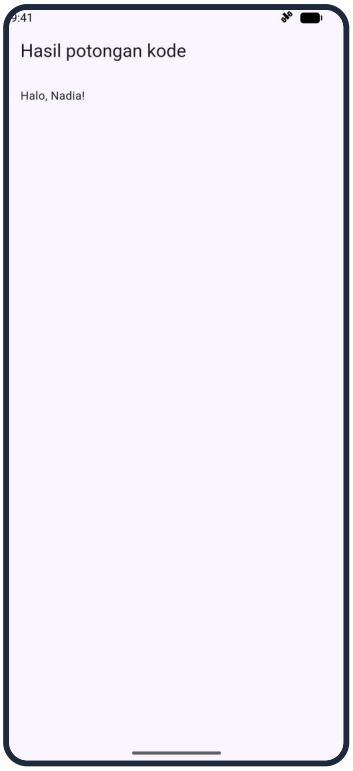
Tanyakan: *"Apakah widget ini perlu mengingat sesuatu yang bisa berubah saat aplikasi berjalan?"* Jika **tidak**, pakai StatelessWidget. Jika **ya** (angka penghitung, status centang, teks yang diketik), pakai StatefulWidget.

Kerangka StatelessWidget

```
class SapaanMahasiswa extends StatelessWidget {
  // Data masuk lewat konstruktor dan bersifat final (tidak bisa diubah)
  final String nama;

  const SapaanMahasiswa({super.key, required this.nama});

  @override
  Widget build(BuildContext context) {
    return Text('Halo, $nama!');
  }
}
```



Widget SapaanMahasiswa dengan nama Nadia

Tiga bagian wajib: **field** `final` untuk data, **konstruktor** `const` dengan `super.key`, dan **method** `build` yang mengembalikan widget.

Contoh lengkap StatelessWidget: kartu profil dosen

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text('Dosen Pengampu')),
        body: ListView(
          padding: const EdgeInsets.all(16),
          children: const [
            // Satu class dipakai berulang dengan data berbeda
            KartuDosen(
              nama: 'Dr. Ratna Dewi, S.Kom., M.Kom.',
              bidang: 'Rekayasa Perangkat Lunak',
```

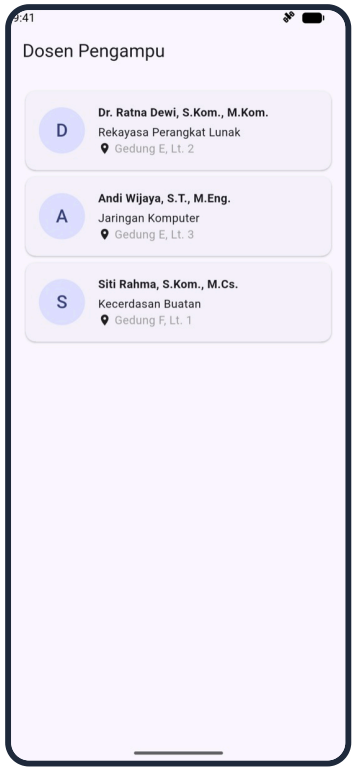
```
              ruang: 'Gedung E, Lt. 2',
            ),
            KartuDosen(
              nama: 'Andi Wijaya, S.T., M.Eng.',
              bidang: 'Jaringan Komputer',
              ruang: 'Gedung E, Lt. 3',
            ),
            KartuDosen(
              nama: 'Siti Rahma, S.Kom., M.Cs.',
              bidang: 'Kecerdasan Buatan',
              ruang: 'Gedung F, Lt. 1',
            ),
          ],
        ),
      ),
    );
  }
}

class KartuDosen extends StatelessWidget {
  final String nama;
  final String bidang;
  final String ruang;

  const KartuDosen({
    super.key,
    required this.nama,
    required this.bidang,
    required this.ruang,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Row(
          children: [
            CircleAvatar(
              radius: 28,
              // Ambil huruf pertama nama sebagai inisial
              child: Text(nama.substring(0, 1), style: const TextStyle(fontSize: 22)),
            ),
            const SizedBox(width: 16),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(nama, style: const TextStyle(fontWeight: FontWeight.bold)),
                  const SizedBox(height: 4),
                  Text(bidang),
                  Row(
                    children: [
                      const Icon(Icons.room, size: 16),
                      const SizedBox(width: 4),
                      Text(ruang, style: const TextStyle(color: Colors.grey)),
                    ],
                  ),
                ],
```

```
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```



Satu class KartuDosen dipakai untuk tiga dosen

Penjelasan kode

- `KartuDosen` tidak menyimpan data yang berubah. Semua datanya datang dari luar (parameter konstruktor), jadi cocok sebagai StatelessWidget.
- Karena konstruktornya `const` dan datanya berupa string literal, daftar `children` di `ListView` bisa diberi `const`.
- `Expanded` mencegah teks nama yang panjang menabrak tepi layar (dibahas di Modul 04).

> ℹ️ **"Stateless" bukan berarti tidak pernah digambar ulang**
>
> StatelessWidget tetap bisa tampil berbeda jika **induknya** membangunnya ulang dengan data baru. Yang tidak bisa adalah mengubah tampilannya **sendiri**. Untuk itu kita perlu StatefulWidget di section berikutnya.

⚠️ **Kesalahan umum**

- Mencoba mengubah field di StatelessWidget (`nama = 'baru'`). Field-nya `final` , jadi akan error.
- Langsung memakai StatefulWidget untuk semua hal "berjaga-jaga". Mulailah dari StatelessWidget, ubah ke Stateful hanya jika memang butuh state.

10. StatefulWidget & setState: Counter Lengkap

`StatefulWidget` terdiri dari dua class: class widget (konfigurasi, tetap immutable) dan class `State` (tempat menyimpan data yang berubah). Untuk mengubah data dan memperbarui layar, kita memanggil `setState()`.

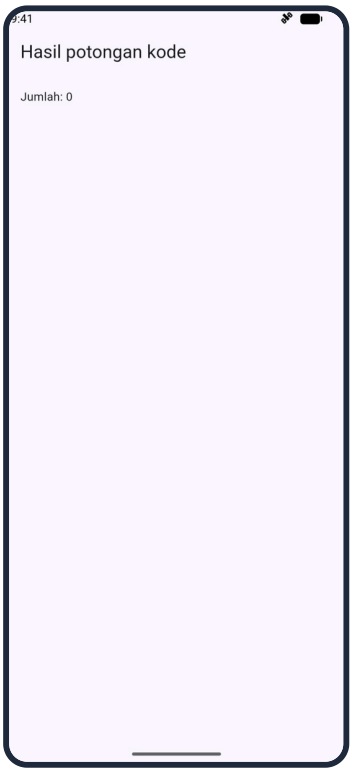
Kerangka StatefulWidget

```
// Class 1: widget-nya. Hanya bertugas membuat objek State.
class Penghitung extends StatefulWidget {
  const Penghitung({super.key});

  @override
  State<Penghitung> createState() => _PenghitungState();
}

// Class 2: State-nya. Di sinilah data yang berubah disimpan.
class _PenghitungState extends State<Penghitung> {
  int _jumlah = 0; // data (state)

  @override
  Widget build(BuildContext context) {
    return Text('Jumlah: $_jumlah');
  }
}
```



Kerangka Penghitung menampilkan nilai awal 0

Tanda garis bawah (`_`) di depan nama berarti *private*: hanya bisa diakses dari file yang sama. Ini konvensi untuk class State.

> 💡 **Pintasan VS Code**
>
> Ketik `stful` lalu tekan `Enter` untuk membuat kerangka StatefulWidget, dan `stless` untuk StatelessWidget.

Contoh lengkap: counter dengan tambah, kurang, dan reset

📄 **`lib/main.dart`**

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.orange),
      home: const HalamanCounter(),
    );
  }
}

class HalamanCounter extends StatefulWidget {
  const HalamanCounter({super.key});

  @override
  State<HalamanCounter> createState() => _HalamanCounterState();
}

class _HalamanCounterState extends State<HalamanCounter> {
  int _hadir = 0; // jumlah mahasiswa yang hadir

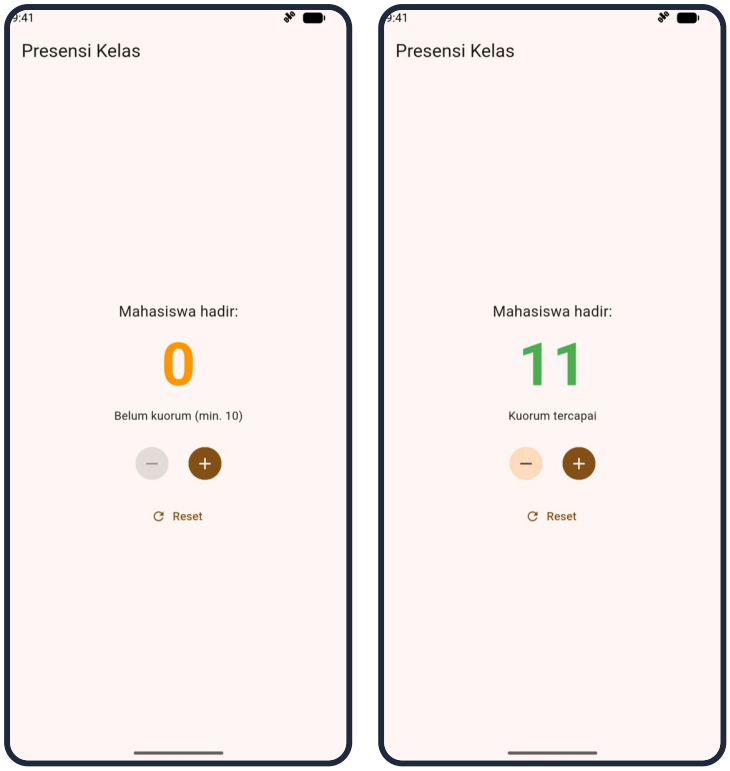
  void _tambah() {
    setState(() {
      _hadir++;
    });
  }

  void _kurang() {
    // Jangan sampai negatif
    if (_hadir == 0) return;
    setState(() {
      _hadir--;
    });
  }
```

```dart
  void _reset() {
    setState(() => _hadir = 0);
  }

  @override
  Widget build(BuildContext context) {
    // build dipanggil ulang setiap setState; print ini membuktikannya
    debugPrint('build() dipanggil, _hadir = $_hadir');

    return Scaffold(
      appBar: AppBar(title: const Text('Presensi Kelas')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Mahasiswa hadir:', style: TextStyle(fontSize: 18)),
            Text(
              '$_hadir',
              style: TextStyle(
                fontSize: 72,
                fontWeight: FontWeight.bold,
                // Warna berubah sesuai state
                color: _hadir >= 10 ? Colors.green : Colors.orange,
              ),
            ),
            Text(_hadir >= 10 ? 'Kuorum tercapai' : 'Belum kuorum (min. 10)'),
            const SizedBox(height: 24),
            Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                IconButton.filledTonal(
                  onPressed: _hadir == 0 ? null : _kurang, // null = tombol nonaktif
                  icon: const Icon(Icons.remove),
                ),
                const SizedBox(width: 16),
                IconButton.filled(
                  onPressed: _tambah,
                  icon: const Icon(Icons.add),
                ),
              ],
            ),
            const SizedBox(height: 16),
            TextButton.icon(
              onPressed: _reset,
              icon: const Icon(Icons.refresh),
              label: const Text('Reset'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Counter presensi: tampilan awal, tombol kurang nonaktif

Setelah tombol tambah ditekan beberapa kali

Apa yang terjadi saat tombol + ditekan?

1. Flutter memanggil `_tambah()`.
2. Di dalam `setState`, nilai `_hadir` bertambah 1.
3. `setState` menandai widget ini sebagai "kotor" (*dirty*), artinya perlu digambar ulang.
4. Pada frame berikutnya, Flutter memanggil ulang `build()`. Karena `_hadir` sudah berubah, `Text` menampilkan angka baru dan warnanya ikut menyesuaikan.

Buka tab **Debug Console** di VS Code: setiap kali tombol ditekan, baris `build() dipanggil` muncul.

Penjelasan kode

- `onPressed: null` membuat tombol nonaktif (abu-abu). Kita memakainya saat nilai sudah 0.
- `setState(() => _hadir = 0)` adalah bentuk singkat (arrow function) dari `setState(() { _hadir = 0; })`.
- Logika yang tidak mengubah tampilan (seperti `if (_hadir == 0) return;`) boleh di luar `setState`.

Contoh kedua: toggle dan input teks

State tidak harus angka. Berikut contoh state bertipe `bool` dan `String`:

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
```

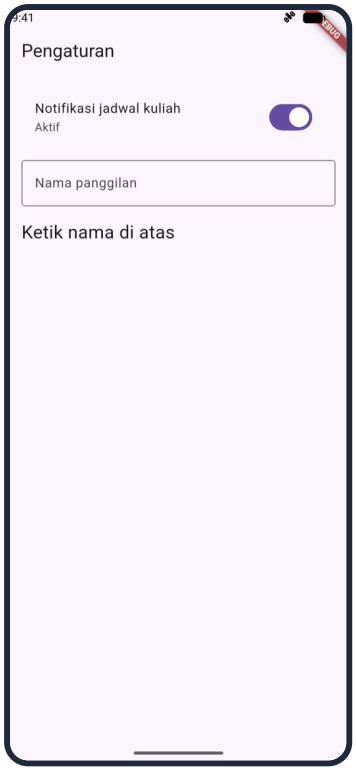
```dart
void main() {
  runApp(const MaterialApp(home: HalamanPengaturan()));
}

class HalamanPengaturan extends StatefulWidget {
  const HalamanPengaturan({super.key});

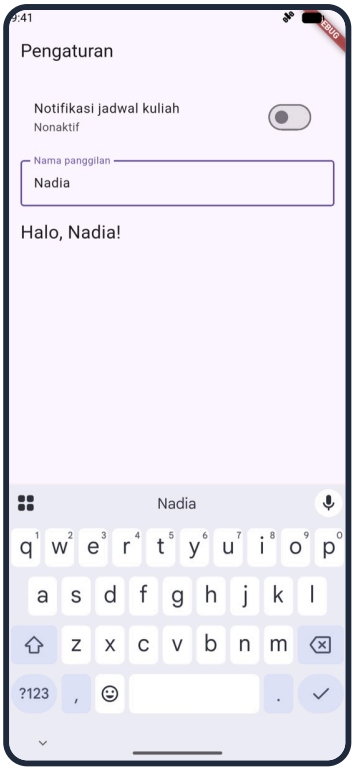
  @override
  State<HalamanPengaturan> createState() => _HalamanPengaturanState();
}

class _HalamanPengaturanState extends State<HalamanPengaturan> {
  bool _notifikasiAktif = true;
  String _nama = '';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          SwitchListTile(
            title: const Text('Notifikasi jadwal kuliah'),
            subtitle: Text(_notifikasiAktif ? 'Aktif' : 'Nonaktif'),
            value: _notifikasiAktif,
            onChanged: (nilaiBaru) {
              setState(() => _notifikasiAktif = nilaiBaru);
            },
          ),
          const SizedBox(height: 16),
          TextField(
            decoration: const InputDecoration(
              labelText: 'Nama panggilan',
              border: OutlineInputBorder(),
            ),
            onChanged: (teks) {
              setState(() => _nama = teks);
            },
          ),
          const SizedBox(height: 16),
          Text(
            _nama.isEmpty ? 'Ketik nama di atas' : 'Halo, $_nama!',
            style: const TextStyle(fontSize: 22),
          ),
        ],
      ),
    );
  }
}
```



SwitchListTile dan TextField dengan state



Setelah nama diketik: teks sapaan langsung berubah

Setiap kali switch digeser atau huruf diketik, `setState` memperbarui variabel dan layar langsung menyesuaikan. Form dibahas lebih dalam di Modul 05.

⚠ **Kesalahan umum**

- **Mengubah variabel tanpa `setState`.** Nilainya berubah di memori, tetapi layar tidak diperbarui.
- **Menaruh variabel state di dalam `build()`.** Contoh: `int _hadir = 0;` ditulis di dalam `build`. Nilainya kembali ke 0 setiap build. Variabel state harus menjadi field class State.
- **Memanggil `setState` di dalam `build()`.** Ini memicu build tanpa henti dan error.
- **Menaruh state di class widget** (yang `extends StatefulWidget`). Class widget harus immutable; state disimpan di class `State`.

11. Lifecycle State dengan Demo Log

Objek `State` punya **siklus hidup** (lifecycle): ia dibuat, dipasang ke pohon, digambar berkali-kali, diperbarui, lalu dibuang. Flutter memanggil method tertentu di setiap tahap. Memahami urutannya adalah kunci menghindari kebocoran memori (memory leak) dan error saat aplikasi berjalan.

```
createState()
     |
     ▼
initState()              ← sekali, saat State pertama kali dibuat
     |
     ▼
didChangeDependencies()  ← setelah initState, dan tiap kali Theme/MediaQuery yang dipakai berubah
     |
     ▼
build()  ◄──────────────┐  ← berkali-kali
     |                  |
     ├── setState() ────┤
     ├── didUpdateWidget()┤ ← induk membangun ulang dengan parameter baru
     |                  |
     ▼
dispose()                ← sekali, saat widget dibuang dari pohon selamanya
```

1. **`createState()`**: Dipanggil saat framework membuat elemen baru di pohon elemen. Isinya hanya membuat objek State.
2. **`initState()`**: Dipanggil tepat **SATU KALI** saat objek state pertama kali dimasukkan ke tree. Cocok untuk inisialisasi controller, controller animasi, timer, atau mengambil data awal.
3. **`didChangeDependencies()`**: Dipanggil setelah `initState()` dan setiap kali objek yang dibaca lewat `context` (seperti `Theme.of` atau `MediaQuery`) berubah.
4. **`build()`**: Mengembalikan struktur widget tree yang akan digambar. Dipanggil setiap kali ada `setState()`, `didUpdateWidget()`, atau perubahan dependensi.
5. **`didUpdateWidget(oldWidget)`**: Dipanggil saat induk membangun ulang widget ini dengan parameter baru. Kita bisa membandingkan `oldWidget` dengan `widget`.
6. **`setState()`**: Memberi sinyal ke framework bahwa ada data internal yang berubah, memicu pemanggilan ulang `build()`.
7. **`dispose()`**: Dipanggil saat widget dihapus secara permanen dari layar. *Wajib menutup stream, timer, AnimationController, dan TextEditingController di sini agar tidak bocor di memori!*

Contoh kecil: pola initState dan dispose

```dart
class KolomNama extends StatefulWidget {
  const KolomNama({super.key});
```

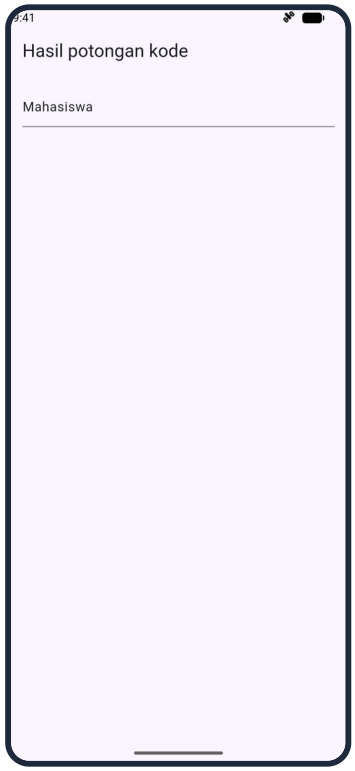
```dart
  @override
  State<KolomNama> createState() => _KolomNamaState();
}

class _KolomNamaState extends State<KolomNama> {
  late final TextEditingController _controller;

  @override
  void initState() {
    super.initState(); // selalu panggil super.initState() PALING AWAL
    _controller = TextEditingController(text: 'Mahasiswa');
  }

  @override
  void dispose() {
    _controller.dispose(); // bersihkan resource milik kita dulu
    super.dispose();       // super.dispose() PALING AKHIR
  }

  @override
  Widget build(BuildContext context) {
    return TextField(controller: _controller);
  }
}
```



TextField dengan controller yang dibuat di initState

Contoh lengkap: demo log lifecycle

Jalankan kode berikut, buka **Debug Console** (VS Code) atau terminal tempat `flutter run` berjalan, lalu tekan tombol-tombolnya satu per satu sambil membaca log yang muncul.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'dart:async';

import 'package:flutter/material.dart';

void main() {
  runApp(const DemoLifecycleApp());
}

/// Induk: menyimpan 3 state yang mempengaruhi anak.
class DemoLifecycleApp extends StatefulWidget {
  const DemoLifecycleApp({super.key});

  @override
  State<DemoLifecycleApp> createState() => _DemoLifecycleAppState();
}

class _DemoLifecycleAppState extends State<DemoLifecycleApp> {
  bool _tampilkanAnak = true; // untuk memicu initState / dispose
  int _nomorPertemuan = 1;    // untuk memicu didUpdateWidget
  bool _modeGelap = false;    // untuk memicu didChangeDependencies

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      // Matikan animasi pergantian tema agar log tidak tercetak berkali-kali
      themeAnimationDuration: Duration.zero,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: _modeGelap ? Brightness.dark : Brightness.light,
      ),
      home: Scaffold(
        appBar: AppBar(title: const Text('Demo Lifecycle')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              FilledButton(
                onPressed: () => setState(() => _tampilkanAnak = !_tampilkanAnak),
                child: Text(_tampilkanAnak ? 'Sembunyikan anak (dispose)' : 'Tampilkan anak
(initState)'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: () => setState(() => _nomorPertemuan++),
                child: const Text('Ganti parameter (didUpdateWidget)'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: () => setState(() => _modeGelap = !_modeGelap),
                child: const Text('Ganti tema (didChangeDependencies)'),
              ),
              const SizedBox(height: 24),
              if (_tampilkanAnak) KartuLifecycle(nomorPertemuan: _nomorPertemuan),
            ],
```

```
          ),
        ),
      ),
    );
  }
}

/// Anak: mencetak log di setiap tahap lifecycle.
class KartuLifecycle extends StatefulWidget {
  final int nomorPertemuan;

  const KartuLifecycle({super.key, required this.nomorPertemuan});

  @override
  // ignore: no_logic_in_create_state
  State<KartuLifecycle> createState() {
    debugPrint('1. createState()');
    return _KartuLifecycleState();
  }
}

class _KartuLifecycleState extends State<KartuLifecycle> {
  Timer? _timer;
  int _detik = 0;

  @override
  void initState() {
    super.initState();
    debugPrint('2. initState()  -> timer dimulai');
    // Timer berjalan tiap detik. WAJIB dibatalkan di dispose().
    _timer = Timer.periodic(const Duration(seconds: 1), (_) {
      setState(() => _detik++);
    });
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    final gelap = Theme.of(context).brightness == Brightness.dark;
    debugPrint('3. didChangeDependencies()  -> tema gelap: $gelap');
  }

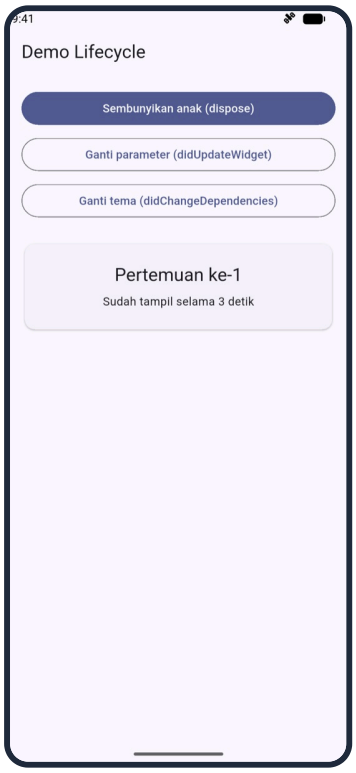
  @override
  void didUpdateWidget(covariant KartuLifecycle oldWidget) {
    super.didUpdateWidget(oldWidget);
    debugPrint(
      '5. didUpdateWidget()  -> pertemuan ${oldWidget.nomorPertemuan} '
      'menjadi ${widget.nomorPertemuan}',
    );
  }

  @override
  void dispose() {
    _timer?.cancel();
    debugPrint('6. dispose()  -> timer dibatalkan setelah $_detik detik');
    super.dispose();
  }

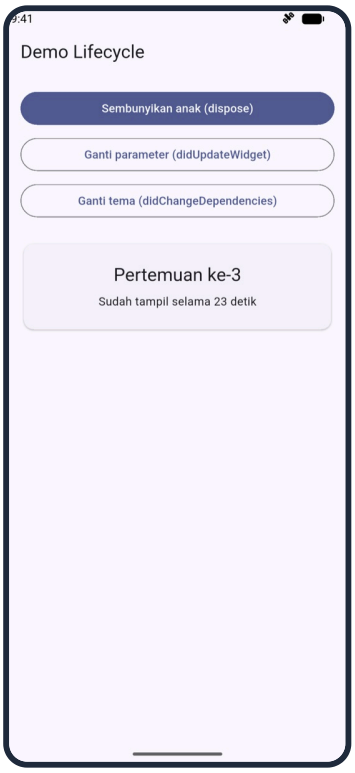
  @override
```

```
  Widget build(BuildContext context) {
    // Dicetak hanya pada build pertama agar log tidak penuh oleh timer
    if (_detik == 0) debugPrint('4. build()');

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            // Akses parameter widget dari State lewat "widget."
            Text(
              'Pertemuan ke-${widget.nomorPertemuan}',
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 8),
            Text('Sudah tampil selama $_detik detik'),
          ],
        ),
      ),
    );
  }
}
```



Demo lifecycle: kartu anak dengan timer



Setelah parameter diganti dua kali (didUpdateWidget)

Contoh log yang muncul

```
// Saat aplikasi dibuka
1. createState()
2. initState()  -> timer dimulai
3. didChangeDependencies()  -> tema gelap: false
4. build()
```

```
// Tekan "Ganti parameter"
5. didUpdateWidget()  -> pertemuan 1 menjadi 2

// Tekan "Ganti tema": induk membangun ulang (didUpdateWidget),
// lalu Theme yang dibaca anak berubah (didChangeDependencies)
5. didUpdateWidget()  -> pertemuan 2 menjadi 2
3. didChangeDependencies()  -> tema gelap: true

// Tekan "Sembunyikan anak"
6. dispose()  -> timer dibatalkan setelah 12 detik

// Tekan "Tampilkan anak" lagi: State BARU dibuat, detik mulai dari 0
1. createState()
2. initState()  -> timer dimulai
3. didChangeDependencies()  -> tema gelap: true
4. build()
```

Penjelasan kode

- `widget.nomorPertemuan`: dari dalam class State, parameter milik widget diakses lewat properti `widget`.
- `initState` tidak boleh memakai `Theme.of(context)` atau `MediaQuery.of(context)`. Hal yang bergantung pada `context` diletakkan di `didChangeDependencies` atau `build`.
- Jika baris `_timer?.cancel()` dihapus, timer tetap berjalan setelah widget hilang dan memanggil `setState` pada State yang sudah dibuang. Hasilnya error *"setState() called after dispose()"*.
- `didUpdateWidget` dipanggil setiap kali induk memanggil `setState`, walaupun nilainya sama (lihat "pertemuan 2 menjadi 2"). Karena itu, di dalamnya biasanya kita membandingkan dulu: `if (oldWidget.nomorPertemuan != widget.nomorPertemuan) { ... }`.
- Saat anak ditampilkan lagi, `_detik` kembali ke 0. Ini membuktikan State lama sudah dibuang dan State baru dibuat.

Pengecekan `mounted` setelah `await`

Jika kamu menunggu proses lama (misalnya request API) lalu memanggil `setState`, cek dulu apakah widget masih terpasang:

```dart
Future<void> _muatData() async {
  await Future.delayed(const Duration(seconds: 2)); // pura-pura request API
  if (!mounted) return; // widget sudah ditutup? jangan setState
  setState(() {
    // perbarui data di sini
  });
}
```

⚠️ **Kesalahan umum**

- Lupa memanggil `super.initState()` di awal atau `super.dispose()` di akhir.
- Membuat controller (`TextEditingController`, `AnimationController`) di dalam `build()`. Controller akan dibuat ulang setiap build dan isinya hilang. Buat di `initState`.
- Tidak membatalkan `Timer` atau `StreamSubscription` di `dispose()`.
- Memanggil `setState` di `dispose()`, atau setelah `await` tanpa cek `mounted`.

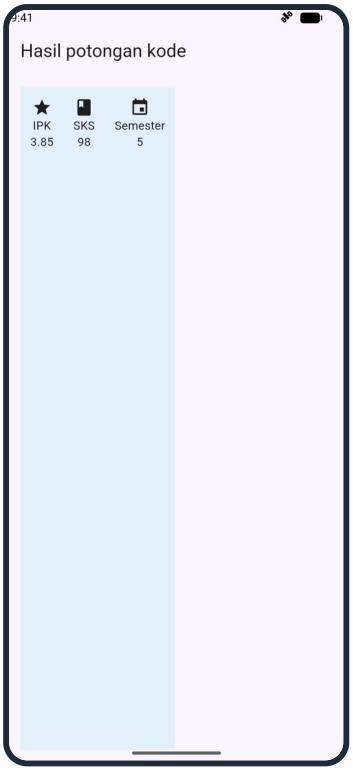
12. Memecah Widget: Parameter & Callback

Method `build()` yang panjangnya ratusan baris sulit dibaca dan sulit diperbaiki. Solusinya: **pecah menjadi widget-widget kecil**. Data dikirim ke widget kecil lewat **parameter konstruktor**, dan widget kecil mengabari induknya lewat **callback** (fungsi yang dikirim sebagai parameter).

Langkah 1: kenali bagian yang berulang

Perhatikan kode "sebelum" berikut. Ada tiga blok `Container` yang hampir sama, hanya berbeda ikon, label, dan nilai:

```
// SEBELUM: kode berulang
Row(
  children: [
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.star), Text('IPK'), Text('3.85')]),
    ),
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.book), Text('SKS'), Text('98')]),
    ),
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.event), Text('Semester'), Text('5')]),
    ),
  ],
)
```



Kode berulang: tiga kotak statistik

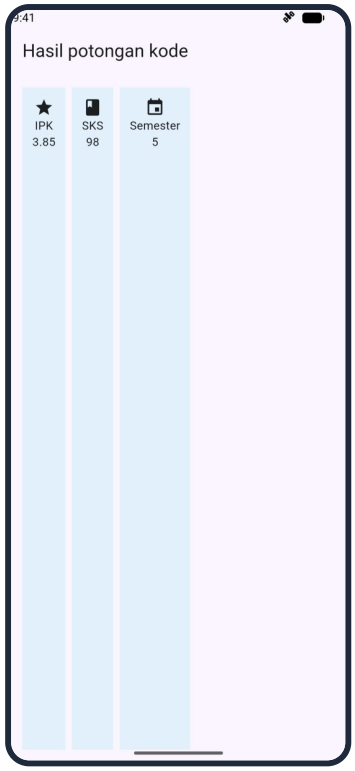
Langkah 2: jadikan class dengan parameter

```
// SESUDAH: satu class, dipakai tiga kali
class KotakStatistik extends StatelessWidget {
  final IconData ikon;
  final String label;
  final String nilai;

  const KotakStatistik({
    super.key,
    required this.ikon,
    required this.label,
    required this.nilai,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: Column(children: [Icon(ikon), Text(label), Text(nilai)]),
    );
  }
}

// Pemakaian:
// Row(children: [
//   KotakStatistik(ikon: Icons.star, label: 'IPK', nilai: '3.85'),
//   KotakStatistik(ikon: Icons.book, label: 'SKS', nilai: '98'),
//   KotakStatistik(ikon: Icons.event, label: 'Semester', nilai: '5'),
// ])
```



Hasil yang sama memakai class KotakStatistik

> 💡 **Refactor otomatis di VS Code**
>
> Letakkan kursor pada nama widget (misalnya `Container`), tekan `Ctrl+.` (macOS: `Cmd+.`), lalu pilih **Extract Widget**. VS Code membuatkan class baru untukmu.

Langkah 3: callback, agar anak bisa "melapor" ke induk

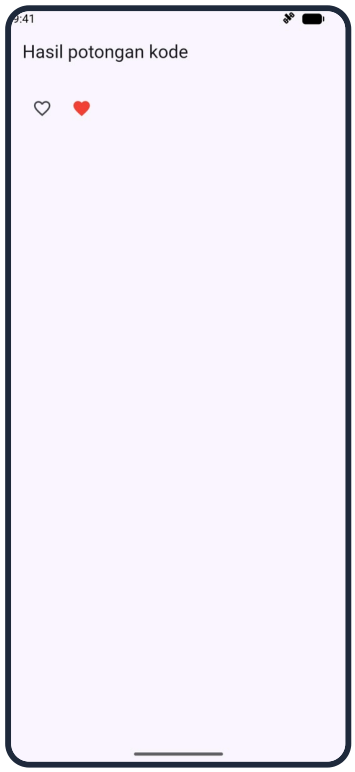
Widget kecil sebaiknya tidak menyimpan state yang juga dibutuhkan induk. Biarkan induk yang menyimpan state, lalu kirim fungsi ke anak. Tipe yang paling sering dipakai:

Tipe	Artinya	Contoh pemakaian
`VoidCallback`	Fungsi tanpa parameter, tanpa nilai kembali	Tombol "Hapus" ditekan
`ValueChanged<T>`	Fungsi yang menerima satu nilai bertipe `T`	Checkbox berubah menjadi `true` / `false`
`void Function(String, int)`	Fungsi dengan parameter bebas	Mengirim nama dan jumlah sekaligus

```dart
class TombolSuka extends StatelessWidget {
  final bool disukai;
  final VoidCallback onTekan; // induk yang menentukan apa yang terjadi

  const TombolSuka({super.key, required this.disukai, required this.onTekan});

  @override
  Widget build(BuildContext context) {
    return IconButton(
      onPressed: onTekan, // teruskan ke induk
      icon: Icon(
        disukai ? Icons.favorite : Icons.favorite_border,
        color: disukai ? Colors.red : null,
      ),
    );
  }
}
```



TombolSuka dalam keadaan belum dan sudah disukai

Contoh lengkap: daftar tugas kuliah yang dipecah menjadi widget kecil

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const HalamanTugas(),
    );
  }
}

/// Model data sederhana untuk satu tugas.
class Tugas {
  final String judul;
  final String matkul;
  bool selesai;

  Tugas({required this.judul, required this.matkul, this.selesai = false});
}

/// INDUK: satu-satunya tempat state disimpan.
class HalamanTugas extends StatefulWidget {
  const HalamanTugas({super.key});

  @override
  State<HalamanTugas> createState() => _HalamanTugasState();
}

class _HalamanTugasState extends State<HalamanTugas> {
  final List<Tugas> _daftar = [
    Tugas(judul: 'Laporan Praktikum 3', matkul: 'Pemrograman Mobile'),
    Tugas(judul: 'ERD Sistem Akademik', matkul: 'Basis Data'),
    Tugas(judul: 'Resume Bab 4', matkul: 'Jaringan Komputer', selesai: true),
  ];

  void _ubahStatus(int index, bool nilai) {
    setState(() => _daftar[index].selesai = nilai);
  }

  void _hapus(int index) {
    setState(() => _daftar.removeAt(index));
  }

  @override
  Widget build(BuildContext context) {
    final jumlahSelesai = _daftar.where((t) => t.selesai).length;

    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Kuliah')),
      body: Column(
        children: [
          // Widget kecil 1: hanya menerima data
          RingkasanTugas(selesai: jumlahSelesai, total: _daftar.length),
          Expanded(
            child: ListView.builder(
              itemCount: _daftar.length,
              itemBuilder: (context, index) {
                // Widget kecil 2: menerima data + 2 callback
                return ItemTugas(
```

```
                  tugas: _daftar[index],
                  onCentang: (nilai) => _ubahStatus(index, nilai),
                  onHapus: () => _hapus(index),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

/// ANAK 1: StatelessWidget yang hanya menampilkan ringkasan.
class RingkasanTugas extends StatelessWidget {
  final int selesai;
  final int total;

  const RingkasanTugas({super.key, required this.selesai, required this.total});

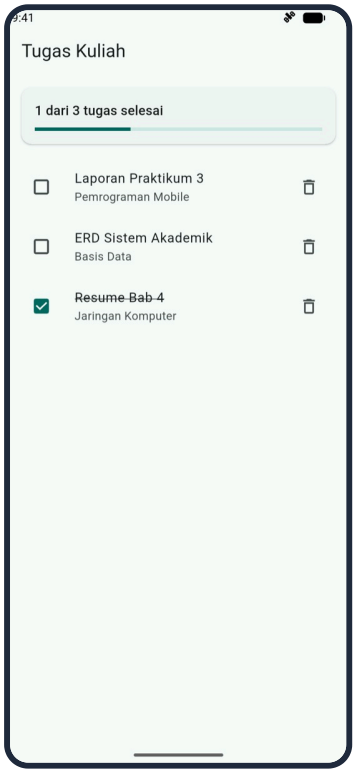
  @override
  Widget build(BuildContext context) {
    final progres = total == 0 ? 0.0 : selesai / total;
    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('$selesai dari $total tugas selesai',
                style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            LinearProgressIndicator(value: progres),
          ],
        ),
      ),
    );
  }
}

/// ANAK 2: menampilkan satu tugas dan melapor ke induk lewat callback.
class ItemTugas extends StatelessWidget {
  final Tugas tugas;
  final ValueChanged<bool> onCentang;
  final VoidCallback onHapus;

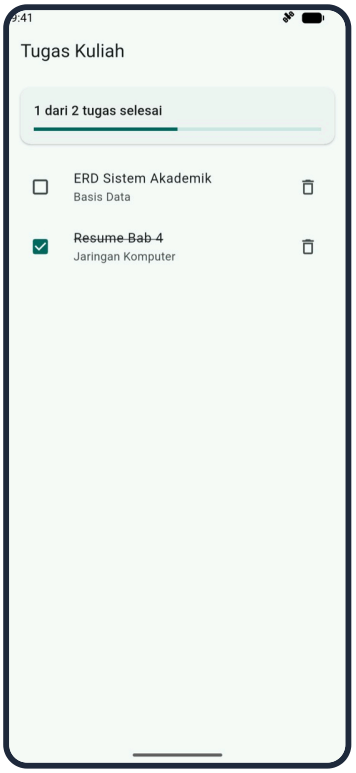
  const ItemTugas({
    super.key,
    required this.tugas,
    required this.onCentang,
    required this.onHapus,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: Checkbox(
        value: tugas.selesai,
```

```
        // Checkbox memberi bool? (bisa null), kita ubah jadi bool
        onChanged: (nilai) => onCentang(nilai ?? false),
      ),
      title: Text(
        tugas.judul,
        style: TextStyle(
          decoration: tugas.selesai ? TextDecoration.lineThrough : null,
        ),
      ),
      subtitle: Text(tugas.matkul),
      trailing: IconButton(
        icon: const Icon(Icons.delete_outline),
        tooltip: 'Hapus',
        onPressed: onHapus,
      ),
    );
  }
}
```



Daftar tugas yang dipecah menjadi widget kecil



Setelah tombol hapus pada item pertama ditekan

Penjelasan alur data

```
_HalamanTugasState (punya state _daftar)
   |  data turun lewat parameter (selesai, total, tugas)
   ▼
RingkasanTugas     ItemTugas
                      |  kejadian naik lewat callback (onCentang, onHapus)
                      ▼
              _ubahStatus / _hapus  →  setState  →  build ulang
```

- **Data turun, kejadian naik.** Ini pola paling penting dalam Flutter. Anak tidak mengubah data sendiri, ia memanggil callback dan induk yang memutuskan.
- `RingkasanTugas` dan `ItemTugas` adalah StatelessWidget walaupun tampilannya berubah, karena state disimpan di induk.
- `(nilai) => _ubahStatus(index, nilai)` adalah fungsi anonim yang "mengingat" `index` item tersebut.

Widget class vs method helper

Kamu mungkin melihat kode yang memecah UI dengan method seperti `Widget _buildHeader() { ... }`. Cara ini boleh untuk potongan kecil, tetapi **class widget lebih disarankan** karena bisa `const`, punya `context` sendiri, dan Flutter bisa melewati build-nya jika datanya tidak berubah.

> ⚠️ **Kesalahan umum**
>
> - **Menyimpan salinan state di anak** (misalnya anak punya `bool _selesai` sendiri). Data induk dan anak jadi tidak sinkron.
> - **Memanggil callback saat build**: `onPressed: onHapus()` (dengan kurung) langsung menjalankan fungsi saat build. Yang benar `onPressed: onHapus` tanpa kurung.
> - **Memecah terlalu kecil** sampai setiap `Text` jadi class. Pecah berdasarkan bagian yang bermakna: header, kartu, item daftar.

13. const & Key

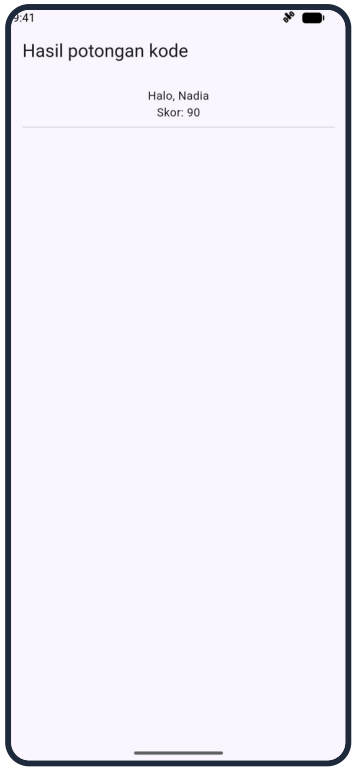
Dua hal kecil yang sering muncul di kode Flutter: kata kunci `const` (supaya widget tidak dibuat ulang tanpa perlu) dan parameter `key` (supaya Flutter bisa mengenali widget yang sama ketika urutannya berubah).

Bagian 1: `const`

Widget yang ditandai `const` dibuat **sekali saat kompilasi** dan dipakai ulang. Saat induknya build ulang, Flutter tahu widget `const` pasti sama sehingga bisa melewatinya. Hasilnya aplikasi lebih hemat kerja.

```
// Boleh const: semua nilainya sudah pasti saat kompilasi
const judul = Text('Jadwal Kuliah');
const jarak = SizedBox(height: 16);
const ikon = Icon(Icons.home, color: Colors.indigo);
const kotak = Padding(padding: EdgeInsets.all(8), child: Text('Halo'));
```

```
// TIDAK boleh const: ada nilai yang baru diketahui saat aplikasi berjalan
Widget contoh(String nama, int skor) {
  return Column(
    children: [
      Text('Halo, $nama'),                  // nama berasal dari variabel
      Text('Skor: $skor'),
      Container(color: Colors.indigo.shade100), // .shade100 bukan konstanta
      // Tetapi anak yang nilainya tetap tetap boleh const:
      const Divider(),
    ],
  );
}
```



Widget dengan nilai dari variabel (tidak bisa const)

Syarat `const`	Contoh
Class punya konstruktor `const`	`const MyWidget({super.key});`
Semua argumen juga konstanta	Literal `'teks'`, angka, `Colors.red`, `Icons.home`, `EdgeInsets.all(8)`
Tidak memakai variabel atau pemanggilan method	Bukan `Text(nama)`, bukan `Theme.of(context)`

> 💡 **Biarkan linter membantu**
>
> Proyek baru Flutter sudah memakai `flutter_lints`. Jika muncul garis biru *"Use 'const' with the constructor to improve performance"*, tekan `Ctrl+.` lalu pilih **Add 'const' modifier**. Perintah `dart fix --apply` bisa menambahkan `const` di seluruh proyek sekaligus.

Bagian 2: `Key`

Flutter mencocokkan widget lama dan baru berdasarkan **tipe dan posisinya**. Biasanya ini cukup. Masalah muncul jika ada beberapa **StatefulWidget bertipe sama** dalam satu daftar lalu

urutannya berubah (dihapus, diurutkan, ditukar). Tanpa key, State bisa "tertinggal" di posisi lama dan menempel ke data yang salah.

Contoh berikut memperlihatkan masalah itu. Setiap kotak menyimpan angka hitungan di State-nya sendiri. Tekan beberapa kotak, lalu tekan tombol **Tukar urutan**.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: DemoKey()));
}

class DemoKey extends StatefulWidget {
  const DemoKey({super.key});

  @override
  State<DemoKey> createState() => _DemoKeyState();
}

class _DemoKeyState extends State<DemoKey> {
  List<String> _nama = ['Andi', 'Budi', 'Citra'];
  bool _pakaiKey = false;

  void _tukar() {
    setState(() => _nama = _nama.reversed.toList());
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Key')),
      body: Column(
        children: [
          SwitchListTile(
            title: const Text('Pakai ValueKey'),
            value: _pakaiKey,
            onChanged: (v) => setState(() => _pakaiKey = v),
          ),
          for (final n in _nama)
            KotakHitung(
              // Tanpa key: State dicocokkan berdasarkan posisi
              // Dengan key: State dicocokkan berdasarkan nama
              key: _pakaiKey ? ValueKey(n) : null,
              nama: n,
            ),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: _tukar,
            icon: const Icon(Icons.swap_vert),
            label: const Text('Tukar urutan'),
          ),
        ],
      ),
    );
  }
```

```dart
}

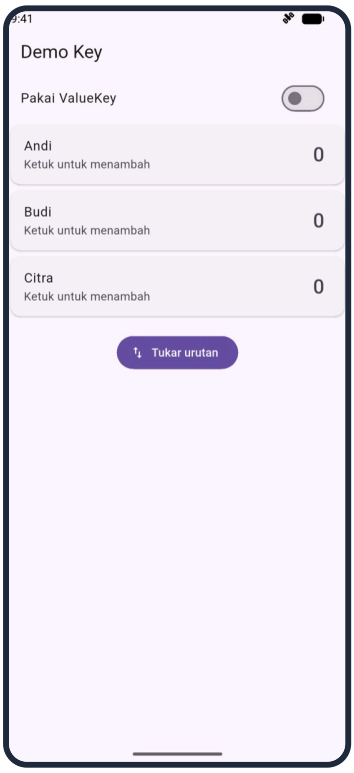
class KotakHitung extends StatefulWidget {
  final String nama;

  const KotakHitung({super.key, required this.nama});

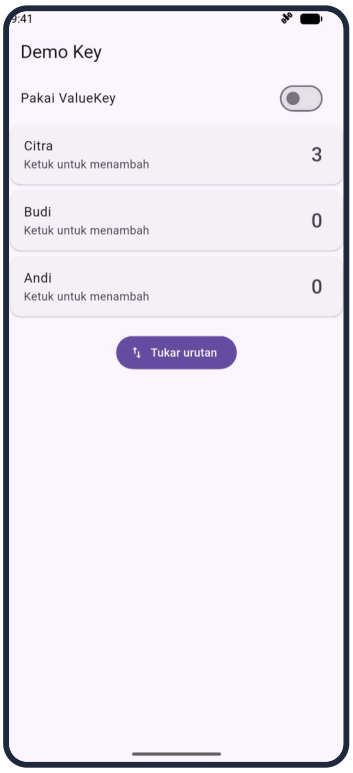
  @override
  State<KotakHitung> createState() => _KotakHitungState();
}

class _KotakHitungState extends State<KotakHitung> {
  int _hitung = 0; // state milik masing-masing kotak

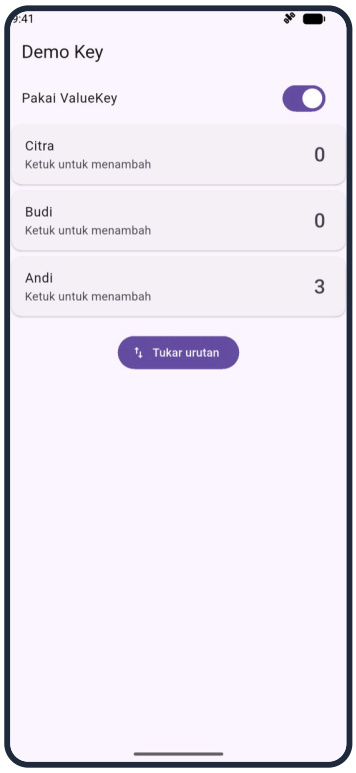
  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(widget.nama),
        subtitle: const Text('Ketuk untuk menambah'),
        trailing: Text('$_hitung', style: const TextStyle(fontSize: 24)),
        onTap: () => setState(() => _hitung++),
      ),
    );
  }
}
```



Tampilan awal demo Key



Tanpa key: Andi diketuk 3x lalu ditukar, angka 3 tertinggal di baris pertama



Dengan ValueKey: angka 3 ikut pindah bersama Andi

Apa yang terjadi?

- **Tanpa key:** ketuk Andi 3 kali (Andi = 3), lalu tukar urutan. Sekarang baris pertama berisi "Citra" tetapi angkanya tetap 3. State tetap di posisi pertama, sementara namanya pindah. Ini

salah.

- **Dengan `ValueKey(n)`:** ulangi langkah yang sama. Angka 3 ikut pindah bersama "Andi" ke baris paling bawah. Flutter mencocokkan State berdasarkan key, bukan posisi.

Jenis key yang umum

Key	Kapan dipakai
`ValueKey(id)`	Item daftar yang punya nilai unik (id, NIM, email). Paling sering dipakai.
`ObjectKey(objek)`	Keunikan ditentukan oleh objeknya sendiri.
`UniqueKey()`	Memaksa widget selalu dianggap baru (State di-reset).
`GlobalKey`	Mengakses State dari luar, misalnya `GlobalKey<FormState>` di Modul 05.

> ⚠️ **Kesalahan umum**
>
> - Memakai **index** sebagai key (`ValueKey(index)`). Index ikut berubah saat urutan berubah, jadi tidak membantu. Pakai id unik dari data.
> - Membuat `UniqueKey()` atau `GlobalKey()` di dalam `build()` . Key baru di setiap build membuat State terus di-reset.
> - Menambahkan `const` pada widget yang argumennya berasal dari variabel. Compiler akan error *"Invalid constant value"*.

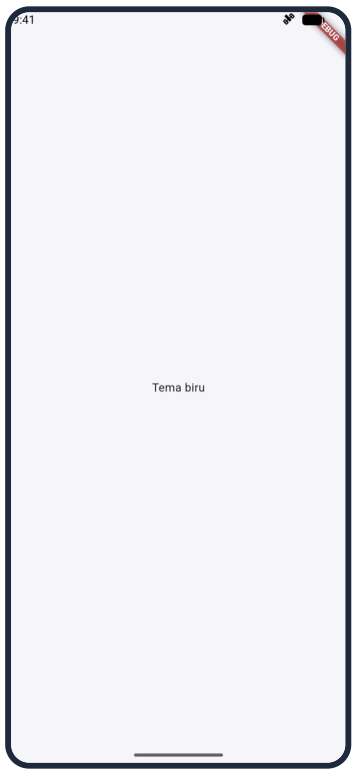
14. ThemeData Material 3 & Mode Gelap

Daripada mengatur warna dan ukuran huruf di setiap widget, Flutter menyediakan `ThemeData`: satu tempat untuk mengatur tampilan seluruh aplikasi. Sejak Flutter 3.16, tema bawaan sudah memakai **Material 3**.

ColorScheme.fromSeed: satu warna, satu palet lengkap

Cukup berikan satu **warna benih** (seed). Flutter menghitung palet lengkap yang serasi: `primary`, `secondary`, `tertiary`, `surface`, `error`, beserta pasangan warna teks di atasnya (`onPrimary`, `onSurface`, dan seterusnya).

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0D47A1)),
  ),
  home: const Scaffold(body: Center(child: Text('Tema biru'))),
)
```



Tema dari ColorScheme.fromSeed

Bentuk singkatnya adalah `ThemeData(colorSchemeSeed: Colors.indigo)`, yang sudah kita pakai di contoh-contoh sebelumnya.

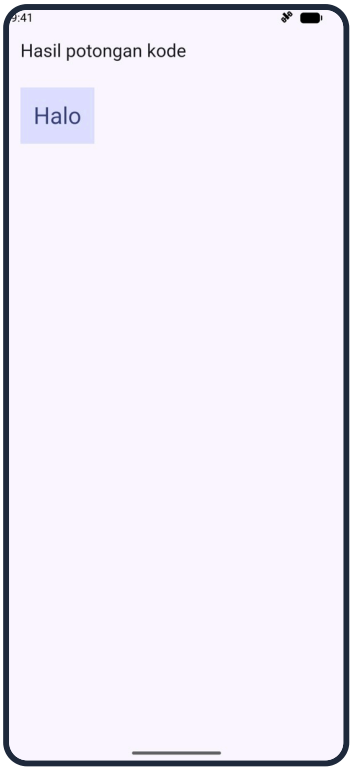
Peran warna di ColorScheme

Warna	Dipakai untuk
`primary` / `onPrimary`	Tombol utama, elemen penting / teks di atasnya.
`primaryContainer` / `onPrimaryContainer`	Latar lembut untuk kartu atau chip yang ditonjolkan.
`secondary`, `tertiary`	Aksen pendukung.
`surface` / `onSurface`	Latar halaman dan kartu / teks umum.
`error`	Pesan kesalahan.

Membaca tema dari widget

```
Widget kotakTema(BuildContext context) {
  final warna = Theme.of(context).colorScheme;
  final teks = Theme.of(context).textTheme;

  return Container(
    padding: const EdgeInsets.all(16),
    color: warna.primaryContainer,
    child: Text(
      'Halo',
      style: teks.headlineMedium?.copyWith(color: warna.onPrimaryContainer),
    ),
  );
}
```



Warna primaryContainer dan teks headlineMedium dari tema

Skala teks Material 3 (`textTheme`)

Kelompok	Nama gaya	Contoh pemakaian
Display	`displayLarge/Medium/Small`	Angka besar, hero
Headline	`headlineLarge/Medium/Small`	Judul halaman
Title	`titleLarge/Medium/Small`	Judul kartu, AppBar
Body	`bodyLarge/Medium/Small`	Paragraf
Label	`labelLarge/Medium/Small`	Teks tombol, chip

Contoh lengkap: tema terang, tema gelap, dan tombol ganti tema

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const AplikasiBertema());
}

class AplikasiBertema extends StatefulWidget {
```

```dart
  const AplikasiBertema({super.key});

  @override
  State<AplikasiBertema> createState() => _AplikasiBertemaState();
}

class _AplikasiBertemaState extends State<AplikasiBertema> {
  // system = ikut pengaturan HP, light = terang, dark = gelap
  ThemeMode _mode = ThemeMode.system;

  static const Color _warnaKampus = Color(0xFFF2A900); // kuning keemasan

  ThemeData _buatTema(Brightness brightness) {
    final skema = ColorScheme.fromSeed(
      seedColor: _warnaKampus,
      brightness: brightness,
    );
    return ThemeData(
      colorScheme: skema,
      // Atur gaya teks tertentu, sisanya ikut bawaan Material 3
      textTheme: const TextTheme(
        headlineSmall: TextStyle(fontWeight: FontWeight.bold),
      ),
      appBarTheme: AppBarTheme(
        backgroundColor: skema.primaryContainer,
        foregroundColor: skema.onPrimaryContainer,
        centerTitle: true,
      ),
      cardTheme: const CardThemeData(
        margin: EdgeInsets.symmetric(vertical: 6),
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: _buatTema(Brightness.light),      // dipakai saat mode terang
      darkTheme: _buatTema(Brightness.dark),   // dipakai saat mode gelap
      themeMode: _mode,                        // menentukan mana yang aktif
      home: HalamanTema(
        mode: _mode,
        onGantiMode: (modeBaru) => setState(() => _mode = modeBaru),
      ),
    );
  }
}

class HalamanTema extends StatelessWidget {
  final ThemeMode mode;
  final ValueChanged<ThemeMode> onGantiMode;

  const HalamanTema({super.key, required this.mode, required this.onGantiMode});

  @override
```

```
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    final teks = Theme.of(context).textTheme;

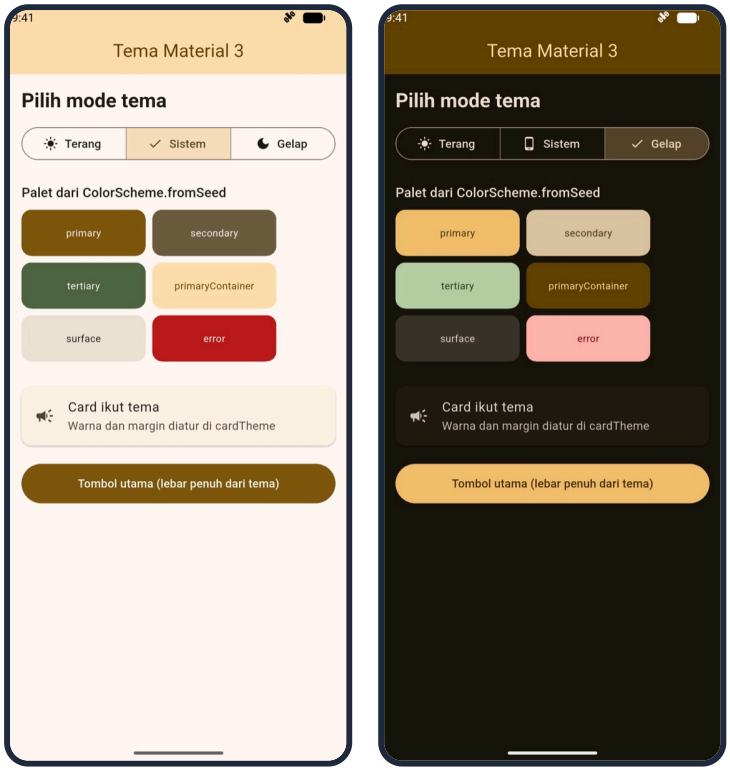
    return Scaffold(
      appBar: AppBar(title: const Text('Tema Material 3')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Pilih mode tema', style: teks.headlineSmall),
          const SizedBox(height: 12),
          SegmentedButton<ThemeMode>(
            segments: const [
              ButtonSegment(value: ThemeMode.light, icon: Icon(Icons.light_mode), label:
Text('Terang')),
              ButtonSegment(value: ThemeMode.system, icon: Icon(Icons.phone_android), label:
Text('Sistem')),
              ButtonSegment(value: ThemeMode.dark, icon: Icon(Icons.dark_mode), label:
Text('Gelap')),
            ],
            selected: {mode},
            onSelectionChanged: (pilihan) => onGantiMode(pilihan.first),
          ),
          const SizedBox(height: 24),
          Text('Palet dari ColorScheme.fromSeed', style: teks.titleMedium),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            runSpacing: 8,
            children: [
              KotakWarna(nama: 'primary', latar: warna.primary, teks: warna.onPrimary),
              KotakWarna(nama: 'secondary', latar: warna.secondary, teks: warna.onSecondary),
              KotakWarna(nama: 'tertiary', latar: warna.tertiary, teks: warna.onTertiary),
              KotakWarna(nama: 'primaryContainer', latar: warna.primaryContainer, teks:
warna.onPrimaryContainer),
              KotakWarna(nama: 'surface', latar: warna.surfaceContainerHighest, teks:
warna.onSurface),
              KotakWarna(nama: 'error', latar: warna.error, teks: warna.onError),
            ],
          ),
          const SizedBox(height: 24),
          const Card(
            child: ListTile(
              leading: Icon(Icons.campaign),
              title: Text('Card ikut tema'),
              subtitle: Text('Warna dan margin diatur di cardTheme'),
            ),
          ),
          const SizedBox(height: 16),
          FilledButton(onPressed: () {}, child: const Text('Tombol utama (lebar penuh dari tema)')),
        ],
      ),
    );
  }
}

class KotakWarna extends StatelessWidget {
  final String nama;
  final Color latar;
```

```
  final Color teks;

  const KotakWarna({super.key, required this.nama, required this.latar, required this.teks});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 150,
      height: 56,
      alignment: Alignment.center,
      decoration: BoxDecoration(color: latar, borderRadius: BorderRadius.circular(12)),
      child: Text(nama, style: TextStyle(color: teks, fontSize: 12)),
    );
  }
}
```



Tema terang Material 3 dengan palet warna

Setelah memilih mode Gelap

Penjelasan kode

- `theme` dan `darkTheme` dibuat dari fungsi yang sama, hanya berbeda `brightness`. Palet gelap otomatis dihitung dari seed yang sama.
- `themeMode` menentukan tema mana yang dipakai. `ThemeMode.system` mengikuti pengaturan mode gelap di HP.
- `appBarTheme`, `cardTheme`, dan `filledButtonTheme` mengatur gaya default semua AppBar, Card, dan FilledButton sekaligus. Itulah sebabnya Card dan tombol di halaman tidak perlu diatur satu per satu.
- Pilihan mode disimpan di `_AplikasiBertemaState` (induk paling atas) dan diubah lewat callback `onGantiMode`, sesuai pola "data turun, kejadian naik".

- `SegmentedButton` adalah tombol pilihan bersegmen dari Material 3.

> ℹ️ **Menyimpan pilihan tema**
>
> Saat aplikasi ditutup, pilihan tema di contoh ini kembali ke "Sistem". Menyimpannya secara permanen akan dipelajari di Modul 10 (SharedPreferences) dan mengelolanya lintas halaman di Modul 08 (GetX).

> ⚠️ **Kesalahan umum**
>
> - **Memberi `brightness` berbeda** antara `ThemeData` dan `ColorScheme`. Atur `brightness` di `ColorScheme.fromSeed` seperti contoh di atas.
> - **Menulis warna langsung** (`Colors.white`, `Colors.black`) di banyak tempat. Di mode gelap teks hitam di latar gelap jadi tidak terbaca. Gunakan `colorScheme.onSurface` dan kawan-kawannya.
> - **Mengisi `darkTheme` tetapi lupa `themeMode`**. Default-nya `ThemeMode.system`, jadi tema gelap hanya muncul jika HP sedang mode gelap.
> - **Memakai `CardTheme`** di `ThemeData.cardTheme`. Versi Flutter terbaru memakai `CardThemeData`.

Tugas Praktikum: Praktikum 3: Kartu Mahasiswa Interaktif dengan Toggle Status Cuti

Sasaran Praktikum

Membuat aplikasi kecil berisi kartu identitas mahasiswa yang dipecah menjadi beberapa widget, memakai StatefulWidget untuk status keaktifan, dan mendukung tema terang/gelap Material 3.

Langkah Pengerjaan

1. Buat project baru `flutter create praktikum_03`, lalu hapus seluruh isi `lib/main.dart`.
2. Atur `MaterialApp` dengan `theme` dan `darkTheme` dari `ColorScheme.fromSeed` memakai satu warna benih pilihanmu, serta `themeMode` yang bisa diganti dari tombol di `AppBar.actions`.
3. Buat `StatefulWidget` bernama `KtmViewerScreen` yang menyimpan `bool _isStatusAktif = true;`.
4. Pecah tampilan menjadi minimal tiga widget kecil: `KartuKtm` (menerima nama, NIM, prodi, status), `BarisInfo` (ikon + label + nilai), dan `TombolStatus` (menerima `VoidCallback onTekan`).
5. Di `KartuKtm`, gunakan `Container` + `BoxDecoration` bergradasi, sudut membulat, dan bayangan (`withValues(alpha: ...)`). Tampilkan foto memakai `CircleAvatar` dengan `NetworkImage` dan ikon cadangan.
6. Tombol status memanggil `setState()` di induk untuk mengubah status (Aktif ↔ Cuti Akademik). Jika Cuti, ubah gradasi kartu menjadi abu-abu dan ikon menjadi peringatan oranye.
7. Di bawah kartu, tampilkan daftar 3 mata kuliah semester ini memakai `Card` + `ListTile` (leading, title, subtitle, trailing).
8. Tambahkan `initState` dan `dispose` pada `KtmViewerScreen` yang mencetak log dengan `debugPrint`, lalu screenshot Debug Console sebagai bukti.
9. Pastikan `flutter analyze` tidak menampilkan error/warning dan gunakan `const` di semua tempat yang memungkinkan.

Output yang Diharapkan

Aplikasi kartu mahasiswa yang rapi, berganti warna dan status saat tombol ditekan, mendukung mode terang/gelap, dan kodenya terpecah menjadi beberapa widget kecil dengan parameter dan callback.

Rubrik Penilaian

Penggunaan StatefulWidget, setState, dan lifecycle yang tepat (30%), Pemecahan widget dengan parameter & callback (25%), Desain kartu dengan Container/BoxDecoration/Card/ListTile (20%), Tema Material 3 & mode gelap (15%), Kerapian kode, const, dan bebas warning analyzer (10%).

Kuis Pemahaman

1. Metode lifecycle manakah pada State yang HANYA dipanggil satu kali saat widget pertama kali diinisialisasi?

A. build()

B. setState()

C. initState()

D. dispose()

2. Mengapa kita tidak boleh memanggil 'setState()' di dalam method 'dispose()'?

A. Karena method dispose() dipanggil saat widget sedang dihancurkan/dihapus dari tree sehingga tidak mungkin me-render ulang lagi

B. Karena compiler Dart akan menghapus seluruh file project

C. Karena dispose() hanya boleh memuat kode styling CSS

D. Karena setState() hanya boleh dipanggil di StatelessWidget

3. Setelah menambahkan folder assets/images/ berisi logo.png, aplikasi menampilkan error 'Unable to load asset'. Penyebab yang paling mungkin adalah...

A. Image.asset hanya mendukung format JPG

B. Folder belum didaftarkan di bagian flutter: assets: pada pubspec.yaml

C. Gambar harus disimpan di dalam folder lib/

D. Perlu menambahkan izin INTERNET di AndroidManifest

4. Apa yang terjadi jika sebuah variabel state diubah TANPA dibungkus setState()?

A. Aplikasi langsung crash

B. Nilai di memori berubah tetapi layar tidak digambar ulang

C. Flutter otomatis memanggil build()

D. Variabel kembali ke nilai awal

5. Menulis Container(color: Colors.red, decoration: BoxDecoration(borderRadius: BorderRadius.circular(8))) menghasilkan error. Bagaimana perbaikannya?

A. Hapus borderRadius

B. Pindahkan color ke dalam BoxDecoration(color: Colors.red, ...)

C. Ganti Container dengan SizedBox

D. Tambahkan const di depan Container

6. Sebuah widget anak perlu memberi tahu induknya bahwa tombol hapus ditekan. Tipe parameter yang paling tepat adalah...

A. String

B. VoidCallback

C. BuildContext

D. Key

7. Daftar StatefulWidget bertipe sama ditukar urutannya, tetapi state (angka hitungan) tertinggal di posisi lama. Solusinya adalah...

A. Memberi setiap item ValueKey berdasarkan id unik datanya

B. Menambahkan const pada setiap item

C. Mengganti semua item menjadi StatelessWidget

D. Memanggil setState dua kali

8. Kode MaterialApp(theme: ..., darkTheme: ..., themeMode: ThemeMode.system). Kapan darkTheme dipakai?

A. Selalu dipakai

B. Tidak pernah dipakai

C. Saat pengaturan perangkat sedang dalam mode gelap

D. Hanya saat aplikasi dijalankan di web

Kunci Jawaban & Pembahasan

1. C — initState() dipanggil tepat satu kali oleh framework ketika objek State dimasukkan ke pohon elemen untuk inisialisasi awal.

2. A — dispose() adalah tahap akhir penghancuran widget dari memori. Memanggil setState() di fase ini akan memicu runtime error karena elemen sudah tidak terpasang di tree.

3. B — Asset harus didaftarkan di pubspec.yaml (di bawah flutter: → assets:), lalu jalankan flutter pub get dan jalankan ulang aplikasi.

4. B — setState() yang memberi tahu framework bahwa widget perlu dibangun ulang. Tanpanya, perubahan nilai tidak terlihat di layar.

5. B — Container tidak boleh menerima color dan decoration sekaligus. Jika memakai decoration, warna ditulis di BoxDecoration.

6. B — VoidCallback adalah fungsi tanpa parameter dan tanpa nilai kembali. Induk mengirim fungsi tersebut, anak memanggilnya saat tombol ditekan (data turun, kejadian naik).

7. A — Tanpa key, Flutter mencocokkan State berdasarkan tipe dan posisi. ValueKey(id) membuat State ikut berpindah bersama datanya.

8. C — ThemeMode.system mengikuti pengaturan sistem. Untuk memaksa, gunakan ThemeMode.dark atau ThemeMode.light.